BỘ GIÁO DỤC VÀ ĐÀO TẠO

TRƯỜNG ĐẠI HỌC MỞ THÀNH PHỐ HỒ CHÍ MINH



BÁO CÁO ĐỒ ÁN NGÀNH

Giảng viên hướng dẫn: ThS. Võ Việt Khoa

Tên đề tài: Công cụ sinh Test Case tự động từ đặc tả yêu cầu bằng AI

Sinh viên thực hiện: Nguyễn Phương Quyên

Mã số sinh viên: *2351050146*

Ngành: Công nghệ thông tin

TP.HCM, ngày 16 tháng 09 năm 2026

NHẬN XÉT CỦA GVHD

MỤC LỤC

CHƯƠNG 1: MỞ ĐẦU 1
1.1. Lý do chọn đề tài 1
1.2. Mục tiêu đề tài 2
1.3. Phạm vi đề tài 3
1.4. Đối tượng sử dụng 4
1.5. Phương pháp và công nghệ sử dụng 4
1.6. Bố cục báo cáo 5
CHƯƠNG 2: KHẢO SÁT HIỆN TRẠNG & CƠ SỞ LÝ THUYẾT 5
2.1. Khảo sát hiện trạng 5
2.1.1. Hiện trạng nghiệp vụ (As-is) 5
2.1.2. Khảo sát sản phẩm trên thị trường 6
2.2. Cơ sở lý thuyết 8
2.2.1. Test Case, Phân vùng tương đương và Phân tích giá trị biên 8
2.2.2. LLM trong sinh Test Case từ yêu cầu 8
2.2.3. Structured Outputs của OpenAI và JSON Schema 9
2.2.4. Prompt engineering 11
2.2.5. Human-in-the-loop và vòng đời Test Case 12
2.2.6. Message Queue và xử lý tác vụ nền 13
2.2.7. Vector Embedding, Cosine Similarity và pgvector 14
2.2.8. Khung đánh giá chất lượng và hiệu quả hệ thống 14
CHƯƠNG 3: PHÂN TÍCH & THIẾT KẾ HỆ THỐNG 15
3.1. Đặc tả yêu cầu chức năng (SRS) 16
3.2. Danh mục quy tắc nghiệp vụ 17
3.3. Đặc tả yêu cầu phi chức năng 18
3.4. Sơ đồ Use Case và đặc tả chi tiết 19
3.4.1. Đặc tả Use Case "Đăng nhập" 21
3.4.2. Đặc tả Use Case "Quản lý người dùng" 22
3.4.3. Đặc tả Use Case "Cấu hình Prompt/Model" 23
3.4.4. Đặc tả Use Case "Tổ chức module và tag" 23
3.4.5. Đặc tả Use Case "Nhập đặc tả yêu cầu" 24
3.4.6. Đặc tả Use Case "Sinh Test Case tự động" 25
3.4.7. Đặc tả Use Case "Rà soát và tối ưu Test Case" 26

3.4.8. Đặc tả Use Case “Duyệt Test Case” 26
3.4.9. Đặc tả Use Case “Xuất file CSV/Excel” 27
3.4.10. Đặc tả Use Case “Quản lý lịch sử phiên bản Test Case” 28
3.4.11. Đặc tả Use Case “Theo dõi Coverage” 29
3.5. Thiết kế luồng nghiệp vụ cốt lõi *30*
3.5.1. Sơ đồ hoạt động 30
3.5.1.1. Sinh Test Case bằng AI 30
3.5.1.2. Rà soát, phát hiện trùng lặp, duyệt và export Test Case 31
3.5.1.3. Cập nhật Requirement theo BR-08 32
3.5.2. Sơ đồ tuần tự 33
3.5.2.1. Sinh Test Case bất đồng bộ bằng AI 33
3.5.2.2. Rà soát và duyệt Test Case 34
3.5.2.3. Phát hiện trùng lặp và merge Test Case 35
3.6. Thiết kế dữ liệu và lớp (ERD & Class Diagram) *36*
3.6.1. Mô hình thực thể ERD 36
3.6.2. Sơ đồ lớp 39
3.7. Thiết kế giao diện *39*
3.8. Ma trận truy vết (Traceability Matrix) *40*
3.9. Thiết kế kiến trúc *43*
CHƯƠNG 4: CÀI ĐẶT & TRIỂN KHAI 45
4.1. *Môi trường & công nghệ* *45*
4.2. Cấu trúc mã nguồn *47*
4.3. Cài đặt database *49*
4.4. Cài đặt các chức năng chính *50*
4.4.1. Xác thực, quản lý người dùng và cấu hình Prompt/Model 51
4.4.2. Module, Requirement và cập nhật theo BR-08 53
4.4.3. Rà soát, duyệt, duplicate và lịch sử phiên bản 54
4.4.4. Coverage, tag và export CSV/XLSX 56
4.4.5. Sinh Test Case bằng AI và theo dõi Generation Job 59
4.5. Phần lõi kỹ thuật đặc thù *59*
4.5.1. Prompt version + Structured Output và retry BR-04 60
4.5.2. Generation Job bất đồng bộ với RabbitMQ/Celery 60
4.5.3. Semantic duplicate bằng pgvector 62
4.6. Đóng gói & triển khai *63*
CHƯƠNG 5: KIỂM THỬ & ĐÁNH GIÁ 65
5.1. Chiến lược kiểm thử *66*

5.2. Thiết kế Test Case *67*
5.3. Kiểm thử tự động & CI *70*
5.4. Ma trận truy vết yêu cầu → Test Case *72*
5.5. Kết quả & defect *74*
5.6. Đánh giá đối chiếu mục tiêu Chương 1 ..... *76*
5.7. Phần chưa kiểm được *77*
CHƯƠNG 6: KẾT LUẬN & HƯỚNG PHÁT TRIỂN 78
6.1. Kết quả đạt được *78*
6.2. Hạn chế *79*
6.3. Hướng phát triển *80*
6.4. Bài học rút ra *81*

DANH MỤC TỪ VIẾT TẮT

Từ viết tắt	Diễn giải
AI	Artificial Intelligence - Trí tuệ nhân tạo
API	Application Programming Interface
BR	Business Rule - Quy tắc nghiệp vụ
BRD	Business Requirements Document
BVA	Boundary Value Analysis - Phân tích giá trị biên
EP	Equivalence Partitioning - Phân vùng tương đương
ERD	Entity Relationship Diagram
HITL	Human-in-the-loop
JWT	JSON Web Token
LLM	Large Language Model
QA	Quality Assurance
RBAC	Role-Based Access Control
RTM	Requirement Traceability Matrix
SRS	Software Requirements Specification
UC	Use Case

DANH MỤC HÌNH VẼ

Hình 3.1 : Sơ đồ Use Case 20
Hình 3.2 : Sơ đồ hoạt động "Sinh Test Case bằng AI" 30
Hình 3.3 : Sơ đồ hoạt động "Rà soát, khử trùng lặp, duyệt và export Test Case" 31
Hình 3.4 : Sơ đồ hoạt động "Update Requirement & đánh dấu Test Case cần rà soát lại" 32
Hình 3.5 : Sơ đồ tuần tự "Sinh Test Case bất đồng bộ bằng AI" 34
Hình 3.6 : Sơ đồ tuần tự "Rà soát và duyệt Test Case" 35
Hình 3.7 : Sơ đồ tuần tự "Phát hiện trùng lặp ngữ nghĩa và merge Test Case" ... 36
Hình 3.8 : Mô hình thực thể ERD 37
Hình 3.9 : Sơ đồ lớp miền sinh và rà soát Test Case 39
Hình 3.10 : Wireframe định hướng màn hình rà soát Test Case 40
Hình 3.11 : Sơ đồ kiến trúc 44
Hình 4.1 : System Config - Prompt / Model version và quản lý user theo role 53
Hình 4.2 : QA mở lại Requirement đã lưu theo Module để cập nhật 54
Hình 4.3 : Màn Review Test Case với HITL và Duplicate & Version 56
Hình 4.4 : Lịch sử phiên bản và khôi phục Test Case 56
Hình 4.5 : Coverage theo Module và điểm vào Export 58
Hình 4.6 : File XLSX export từ bộ Test Case APPROVED 58
Hình 4.7 : Generation Job hoàn tất và Test Case được sinh chờ review 61
Hình 4.8 : Tạo Test Case thủ công và lưu ở trạng thái DRAFT 62
Hình 4.9 : Health check xác nhận application, database và queue hoạt động 65
Hình 5.1 : GitHub Actions run #91 – baseline 108 backend test PASS 71
Hình 5.2 : Kết quả Newman API smoke Tuần 09 72

DANH MỤC BẢNG

Bảng 1.1 : Vấn đề hiện trạng và năng lực hệ thống giải quyết 2
Bảng 1.2 : Mục tiêu đo lường của đề tài 3
Bảng 2.1 : So sánh TestRail, Katalon, testRigor và hệ thống đề xuất 7
Bảng 2.2 : Ví dụ minh họa BVA trên một Requirement có miền 1-8 8
Bảng 2.3 : Cấu trúc Test Case chuẩn của hệ thống 11
Bảng 2.4 : Vòng đời trạng thái Test Case trong mô hình HITL 13
Bảng 2.5 : Khung chỉ số đánh giá chất lượng và hiệu quả hệ thống 15
Bảng 3.1 : Đặc tả yêu cầu chức năng của hệ thống 17
Bảng 3.2 : Danh mục quy tắc nghiệp vụ BR-01 đến BR-09 [1] 18
Bảng 3.3 : Yêu cầu phi chức năng có tiêu chí kiểm được 19
Bảng 3.4 : Danh mục Use Case chính 21
Bảng 3.5 : Cấu trúc dữ liệu đề xuất 38
Bảng 3.6 : Ma trận truy vết mở rộng 43
Bảng 3.7 : Thành phần kiến trúc đề xuất 45
Bảng 4.1 : Môi trường và phiên bản công nghệ của hệ thống 46
Bảng 4.2 : Trách nhiệm các khu vực mã nguồn và ánh xạ kiến trúc 48
Bảng 4.3 : Ánh xạ ERD sang schema PostgreSQL hiện hành 50
Bảng 4.4 : Truy vết chức năng triển khai sang UC/YC/BR 51
Bảng 4.5 : Đầu vào / đầu ra cho nhóm UC01 - UC03 52
Bảng 4.6 : Đầu vào/đầu ra của Requirement và BR-08 53
Bảng 4.7 : Đầu vào/đầu ra cho HITL, version và duplicate 55
Bảng 4.8 : Đầu vào/đầu ra của Coverage và Export 57
Bảng 4.9 : Khuôn năm phần cho UC06 – Sinh Test Case bằng AI 59
Bảng 4.10 : Biến môi trường cần cho triển khai 64
Bảng 4.11 : Tài khoản demo sau khi chạy seed 64
Bảng 5.1 : Chiến lược, môi trường và tiêu chí dừng kiểm thử 66
Bảng 5.2 : Test Case rút gọn và kết quả thực tế 68
Bảng 5.3 : Test Case đủ 9 trường – TC-T09-DF-001 68
Bảng 5.4 : Test Case đủ 9 trường – TC-T09-DF-002 69
Bảng 5.5 : Test Case đủ 9 trường – TC-T09-DF-003 70
Bảng 5.6 : Kết quả kiểm thử tự động, API smoke và CI 70
Bảng 5.7 : Ma trận truy vết YC/BR → Test Case → kết quả gần nhất 74
Bảng 5.8 : So sánh hai lượt kiểm thử tổng thể 74
Bảng 5.9 : Pass/Fail hai lượt cho bộ regression defect Tuần 09 74
Bảng 5.10 : Danh sách defect gắn GitHub Issue và regression 75
Bảng 5.11 : Thống kê defect theo mức nghiêm trọng tại thời điểm chốt báo cáo 75
Bảng 5.12 : Đối chiếu KPI Chương 1 với số đo thực tế 77
Bảng 5.13 : Kết quả Gold Set 20 Requirement 77
Bảng 5.14 : Phần chưa kiểm được và kế hoạch kiểm tiếp 78
Bảng 6.1 : Hạn chế, nguyên nhân và ảnh hưởng 80
Bảng 6.2 : Hướng phát triển gắn với hạn chế 81

CHƯƠNG 1: MỞ ĐẦU

1.1. Lý do chọn đề tài

Trong quy trình đảm bảo chất lượng phần mềm, việc chuyển đặc tả yêu cầu thành các ca kiểm thử có thể thực thi và truy vết được là một công việc tốn nhiều thời gian, đặc biệt khi yêu cầu được viết bằng ngôn ngữ tự nhiên và thay đổi qua nhiều phiên bản. BRD của đề tài đồ án mô tả trực tiếp các vấn đề hiện trạng: kỹ sư QA phải diễn giải yêu cầu thủ công, dễ bỏ sót điều kiện biên, độ bao phủ không đồng đều giữa các module, khó quản lý phiên bản và khó truy vết Test Case về yêu cầu nguồn [1].

Các nghiên cứu về sinh Test Case từ yêu cầu cũng cho thấy đây không phải vấn đề mới. Tổng quan hệ thống của Mustafa và cộng sự ghi nhận nhiều phương pháp tiền-LLM dựa trên UML, NLP và mô hình hình thức, nhưng việc xử lý yêu cầu tự nhiên vẫn gặp khó khăn khi đặc tả mơ hồ hoặc thiếu cấu trúc [2]. Khi LLM phát triển, khả năng hiểu văn bản tự nhiên mở ra hướng sinh Test Case trực tiếp từ Requirement; tuy nhiên các nghiên cứu gần đây vẫn ghi nhận hạn chế về tính nhất quán, độ đúng và khả năng bỏ sót trường hợp kiểm thử [3][4]. Một công cụ web dùng LLM agent và prompt engineering để sinh test scenario từ user Requirement cũng đã được thử nghiệm, cho thấy tính khả thi của hướng tiếp cận này [5].

Vì vậy, đề tài không đặt mục tiêu thay thế kỹ sư QA bằng AI. Trọng tâm là xây dựng một quy trình hỗ trợ: AI sinh bản nháp có cấu trúc, sau đó con người rà soát, chỉnh sửa, duyệt và chỉ đưa vào sử dụng khi đã đạt yêu cầu. Cách tiếp cận này bám sát mô hình human-in-the-loop trong BRD, đồng thời kết hợp truy vết yêu cầu, quản lý phiên bản và nhật ký hoạt động để giảm rủi ro sử dụng trực tiếp đầu ra AI [1].

Bảng 1.1 tổng hợp các vấn đề của quy trình hiện tại, hệ quả đối với hoạt động kiểm thử và năng lực tương ứng mà hệ thống đã đề xuất để giải quyết.

Vấn đề hiện trạng	Hệ quả	Năng lực hệ thống đề xuất	Nguồn
Viết Test Case thủ công từ Requirement	Tốn thời gian, phụ thuộc kinh nghiệm cá nhân	Sinh Test Case nháp bằng LLM từ SRS/AC	[1][2][3]
Dễ bỏ sót điều kiện biên	Thiếu negative/edge case quan trọng	Prompt yêu cầu BVA/EP và checklist rà soát	[1]
Test Case rời rạc, khó truy vết	Khó biết Test Case đang kiểm yêu cầu nào	Mỗi Test Case gắn module và yêu cầu nguồn	[1]
AI có thể sinh sai hoặc thiếu	Rủi ro dùng nhầm đầu ra chưa kiểm chứng	Human-in-the-loop: Nháp → Rà soát → Duyệt → Export	[1][3][4]
Đầu ra AI dễ không đồng nhất	Khó lưu DB và export theo mẫu	Structured Outputs + JSON Schema + validation	[1]

Bảng 1.1: Vấn đề hiện trạng và năng lực hệ thống giải quyết

1.2. Mục tiêu đề tài

BRD nêu các tiêu chí thành công theo hướng định tính như giảm thời gian soạn Test Case, tăng độ bao phủ, bảo đảm mọi Test Case đưa vào sử dụng đều được con người duyệt, có khả năng export và truy vết về yêu cầu nguồn [1]. Để phục vụ đánh giá tại Chương 5, báo cáo chuyển các tiêu chí này thành các chỉ số đo lường. Các ngưỡng trong Bảng 1.2 dưới đây là mục tiêu thử nghiệm đề xuất cho đồ án, không phải số liệu do BRD cam kết. Chỉ số nào chưa thu thập đủ dữ liệu thực nghiệm sẽ được ghi rõ là "chưa đo được" tại Mục 5.7, không suy diễn kết quả. Bảng 1.2 trình bày các KPI và phương pháp thu thập dữ liệu tương ứng.

KPI	Công thức đo	Mục tiêu thử nghiệm	Cách lấy dữ liệu
Tỉ lệ đầu ra đúng cấu trúc	Số response hoàn tất, không refusal, parse đúng schema / tổng response hợp lệ × 100%	≥ 95%	Log generation + Pydantic validation
Tỉ lệ Test Case được chấp nhận	Số Test Case được duyệt với tối đa chỉnh sửa nhỏ / tổng Test Case AI sinh × 100%	≥ 80%	Audit log + version diff
Giảm thời gian soạn Test Case	(Thời gian thủ công - thời gian dùng hệ thống) / thời gian thủ công × 100%	≥ 50%	Benchmark cùng một bộ Requirement
Độ phủ điều kiện biên	Số boundary condition có Test Case / tổng boundary condition trong bộ mẫu vàng × 100%	≥ 90%	Checklist BVA/EP do QA gán nhãn
Truy vết yêu cầu	Số Test Case có Requirement nguồn / tổng Test Case × 100%	100%	DB constraint + RTM
Export hợp lệ	Số file CSV/XLSX mở được đúng cột / tổng lần export × 100%	100%	Test export

Bảng 1.2: Mục tiêu đo lường của đề tài

1.3. Phạm vi đề tài

Trong phạm vi: nhập/dán đặc tả yêu cầu và Acceptance Criteria; sinh Test Case có cấu trúc bằng OpenAI API; gợi ý điều kiện biên; rà soát, chỉnh sửa, khử trùng lặp, duyệt; tổ chức theo module; export CSV/Excel; quản lý phiên bản; đăng nhập, phân quyền; audit log; cấu hình prompt; thống kê số lượng Test Case và mức độ bao phủ Requirement theo module [1].

Ngoài phạm vi: không huấn luyện/fine-tune mô hình từ đầu; không tự động thực thi Test Case; không đo code Coverage; không sinh source code của hệ thống được kiểm thử; không tích hợp hai chiều sâu với Jira/TestRail; không phát triển native mobile app [1].

1.4. Đối tượng sử dụng

- **Kỹ sư QA/Kiểm thử:** nhập Requirement, sinh Test Case, rà soát, chỉnh sửa, khử trùng lặp và thực hiện các thao tác được cấp quyền.
- **Quản lý kiểm thử (Manager):** quản lý module, theo dõi Coverage, rà soát, duyệt và export Test Case theo quyền.
- **Quản trị hệ thống (Admin):** quản lý người dùng, vai trò, cấu hình prompt/model của hệ thống.

Ba vai trò đăng nhập được triển khai trong hệ thống là **QA**, **Manager** và **Admin**. BA/nhóm phát triển được xem là bên liên quan cung cấp Requirement/Acceptance Criteria hoặc tiếp nhận bộ Test Case đã duyệt, không phải role đăng nhập riêng trong phiên bản hiện tại. Quyền của từng role được kiểm tra tại backend và được mô hình hóa trong Use Case, RBAC và ma trận truy vết ở Chương 3 [1].

1.5. Phương pháp và công nghệ sử dụng

Đề tài áp dụng phương pháp human-in-the-loop: AI đảm nhiệm khâu tạo bản nháp và gợi ý Test Case, còn quyết định rà soát, chỉnh sửa, duyệt hoặc loại bỏ thuộc về người dùng có quyền. Lõi AI sử dụng OpenAI API kết hợp Structured Outputs để sinh dữ liệu theo cấu trúc được backend kiểm tra bằng Pydantic. Prompt và model được quản lý thông qua cấu hình của hệ thống. Các thông tin nhạy cảm như API key, JWT secret và chuỗi kết nối cơ sở dữ liệu được cung cấp qua biến môi trường, không hard-code trong source.

Backend được xây dựng bằng FastAPI, Pydantic, SQLAlchemy và Alembic; dữ liệu được lưu trên PostgreSQL kết hợp pgvector phục vụ phát hiện Test Case tương tự. Frontend sử dụng Next.js, React và TypeScript. Các tác vụ sinh Test Case bằng AI được xử lý bất đồng bộ qua RabbitMQ và Celery. GitHub Actions được sử dụng cho Continuous Integration, bao gồm kiểm tra format/lint, migration, backend test và frontend build. Phiên bản hiện tại chạy trên môi trường local. Việc chọn OpenAI là

quyết định kỹ thuật của đồ án, phù hợp với BRD vì tài liệu chỉ yêu cầu dùng API mô hình có sẵn, không bắt buộc nhà cung cấp cụ thể [1].

1.6. Bố cục báo cáo

- **Chương 1 - Mở đầu:** bối cảnh, mục tiêu, phạm vi, đối tượng sử dụng và công nghệ.
- **Chương 2 - Khảo sát hiện trạng và cơ sở lý thuyết:** hiện trạng QA, sản phẩm thị trường, LLM, BVA/EP, prompt engineering, Structured Outputs/JSON Schema và HITL.
- **Chương 3 - Phân tích và thiết kế hệ thống:** SRS, BR, NFR, Use Case, Activity, Sequence, ERD/Class, wireframe, traceability và kiến trúc.
- **Chương 4 - Cài đặt và triển khai:** trình bày môi trường, công nghệ; cấu trúc source; cơ sở dữ liệu, migration; cách cài đặt các chức năng theo UC/YC/BR; phần lõi kỹ thuật của hệ thống; quy trình cài đặt, chạy lại và bàn giao.
- **Chương 5 - Kiểm thử và đánh giá:** trình bày chiến lược kiểm thử, thiết kế Test Case, kiểm thử tự động và CI, ma trận truy vết, kết quả 2 lượt chạy và defect, đánh giá KPI cùng các phần chưa kiểm được.
- **Chương 6 - Kết luận và hướng phát triển:** tổng hợp kết quả đạt được bằng số liệu từ Chương 5, các hạn chế, hướng phát triển tương ứng và bài học rút ra.

CHƯƠNG 2: KHẢO SÁT HIỆN TRẠNG & CƠ SỞ LÝ THUYẾT

2.1. Khảo sát hiện trạng

2.1.1. Hiện trạng nghiệp vụ (As-is)

Theo BRD, trước khi có hệ thống hỗ trợ, kỹ sư QA tiếp nhận Requirement và Acceptance Criteria, tự đọc và diễn giải nội dung nghiệp vụ, sau đó xây dựng Test Case thủ công bằng bảng tính hoặc tài liệu văn bản. Người kiểm thử phải tự xác định happy path, negative case, điều kiện biên, mức ưu tiên và liên kết Test Case với yêu cầu nguồn. Khi Requirement thay đổi, việc rà soát Test Case bị ảnh hưởng cũng chủ yếu phụ thuộc vào người thực hiện.

Có thể khái quát quy trình hiện tại thành các bước: tiếp nhận Requirement → phân tích điều kiện và quy tắc → viết Test Case thủ công → rà soát và cập nhật khi Requirement thay đổi. Cách làm này tốn thời gian, phụ thuộc kinh nghiệm cá nhân, dễ bỏ sót điều kiện biên, độ bao phủ không đồng đều giữa các module và gây khó khăn cho quản lý phiên bản cũng như truy vết Test Case về Requirement nguồn [1]. Đây cũng là nhóm vấn đề thường được đề cập trong các nghiên cứu về Requirement-based test generation, nơi yêu cầu ngôn ngữ tự nhiên cần được chuyển thành các biểu diễn có thể kiểm thử [2].

2.1.2. Khảo sát sản phẩm trên thị trường

Báo cáo khảo sát ba nhóm công cụ đại diện: **TestRail, Katalon và testRigor**. Thông tin trong phần này ưu tiên tài liệu chính thức của nhà cung cấp, được kiểm tra lại vào tháng 08/2026.

TestRail đã có chức năng AI tạo Test Case từ Requirement, cho phép người dùng rà soát, chỉnh sửa, chọn Test Case trước khi tạo đầy đủ bước và expected result; tài liệu hỗ trợ cũng mô tả quyền truy cập AI và audit log cho các thao tác liên quan [6]. Điểm mạnh của TestRail là quản lý Test Case, traceability và workflow đã trưởng thành. Tuy nhiên, đây là nền tảng quản lý kiểm thử rộng; báo cáo không tìm thấy trong tài liệu công khai cơ chế để sinh viên tự định nghĩa JSON Schema làm contract kỹ thuật cho chính LLM output của hệ thống.

Katalon True Platform có AI-assisted Test Case generator có thể sinh Test Case/test step từ Requirements; tài liệu chính thức yêu cầu người dùng rà soát nội dung AI trước khi lưu/approve [7]. Katalon phù hợp cho một nền tảng chất lượng toàn diện, nhưng đề tài đồ án tập trung hẹp vào luồng *Requirement → structured Test Case → HITL → export* và cần toàn quyền kiểm soát schema, prompt, dữ liệu lưu trữ và truy vết.

testRigor tập trung vào test automation bằng plain English và có thể chuyển Test Case/instruction thành kịch bản tự động hóa [8]. Vì mục tiêu của đề tài không phải thực thi automation test mà là sinh và quản lý high-level/manual Test Case từ Requirement, testRigor được xem như sản phẩm tham khảo về AI-assisted testing nhưng không cùng phạm vi cốt lõi.

Từ các sản phẩm đã khảo sát, Bảng 2.1 đối chiếu những năng lực liên quan trực tiếp đến phạm vi đề tài và làm cơ sở xác định khoảng trống mà hệ thống đề xuất tập trung giải quyết.

Tiêu chí	TestRail	Katalon	testRigor	Hệ thống đề xuất
Sinh Test Case từ Requirement	Có [6]	Có [7]	Không phải trọng tâm [8]	Có - chức năng lõi
Human review	Có [6]	Có [7]	Có thể chỉnh instruction	Bắt buộc theo BR-01
JSON Schema do hệ thống tự quản lý	Không thấy công bố trong tài liệu khảo sát	Không thấy công bố trong tài liệu khảo sát	Không phải trọng tâm	Có - BR-02/BR-04
Prompt/model cấu hình trong chính hệ thống	Không phải mục tiêu chính	Không phải mục tiêu chính	Không phải mục tiêu chính	Có - NC-09
Traceability Requirement ↔ Test Case	Mạnh	Có trong platform	Không phải trọng tâm	Bắt buộc theo BR-03
Version/Audit	Có	Có trong platform	Phụ thuộc sản phẩm	Có - NC-08/NC-11/BR-06
Export CSV/Excel	Có	Có	Không phải trọng tâm	Có - NC-07
Mục tiêu phạm vi	Test management	Software quality platform	Test automation	Tool chuyên biệt cho Requirement-to-test-case

Bảng 2.1: So sánh TestRail, Katalon, testRigor và hệ thống đề xuất

Khoảng trống mà đề tài lựa chọn không phải là tuyên bố "thị trường chưa có AI sinh Test Case". Thực tế TestRail và Katalon đã có chức năng tương tự [6][7]. Giá trị của đồ án nằm ở việc xây một hệ thống nhỏ, minh bạch và có thể kiểm chứng từ đầu

đến cuối: schema tự định nghĩa, output được validate, mỗi Test Case có Requirement nguồn, trạng thái HITL rõ ràng, phiên bản/audit có thể truy vết và toàn bộ quy tắc được kiểm thử trong source code theo SRS/BRD [1].

2.2. Cơ sở lý thuyết

2.2.1. Test Case, Phân vùng tương đương và Phân tích giá trị biên

ISTQB CTFL v4.0.1 cung cấp hệ thuật ngữ và kỹ thuật nền cho kiểm thử hộp đen, trong đó Equivalence Partitioning (EP) chia miền đầu vào thành các lớp có hành vi tương đương, còn Boundary Value Analysis (BVA) tập trung vào giá trị tại và sát biên [9]. Đây là nền tảng để prompt của hệ thống yêu cầu AI không chỉ sinh happy path mà còn phải gợi ý negative và boundary case theo NC-03/BR-09 [1].

Giá trị	Phân loại BVA	Kết quả mong đợi
0	Dưới biên (min - 1)	Không hợp lệ - hệ thống từ chối xử lý
1	Biên dưới	Hợp lệ
2	Lân cận biên dưới	Hợp lệ
7	Lân cận biên trên	Hợp lệ
8	Biên trên	Hợp lệ
9	Trên biên (max + 1)	Không hợp lệ - vượt giới hạn

Bảng 2.2: Ví dụ minh họa BVA trên một Requirement có miền 1-8

Ví dụ trong Bảng 2.2 là một Requirement giả lập dùng để minh họa cách phát hiện giá trị biên, không phải quy tắc nghiệp vụ nội tại của hệ thống. Với miền hợp lệ từ 1 đến 8, người kiểm thử không chỉ chọn một giá trị giữa miền mà còn chú ý cả hai cụm nhạy cảm {0,1,2} và {7,8,9}. Trong hệ thống đề tài, BVA/EP được sử dụng theo NC-03 và BR-09 để định hướng AI gợi ý Test Case từ các ràng buộc có trong Requirement nguồn; kết quả AI vẫn phải được QA rà soát trước khi duyệt [1][9].

2.2.2. LLM trong sinh Test Case từ yêu cầu

LLM có khả năng xử lý Requirement dạng prose và sinh nội dung kiểm thử ở mức ngôn ngữ tự nhiên. Tổng quan của Celik và Mahmoud phân nhóm các hướng

dùng LLM cho test generation và ghi nhận cả cơ hội lẫn hạn chế về tính nhất quán, correctness và lỗi đầu ra [3]. Nghiên cứu của Masuda và cộng sự tập trung trực tiếp vào sinh high-level Test Case từ Requirement document, cho thấy khả năng áp dụng thực tế nhưng vẫn cần phương pháp đánh giá và rà soát [4]. Công cụ của Sami và cộng sự cũng minh họa kiến trúc web-based LLM + prompt engineering để sinh test scenario từ user Requirement [5].

2.2.3. Structured Outputs của OpenAI và JSON Schema

OpenAI Structured Outputs cho phép yêu cầu đầu ra tuân theo JSON Schema do ứng dụng định nghĩa. Tài liệu OpenAI hiện khuyến nghị Structured Outputs thay cho JSON mode khi model hỗ trợ; SDK Python có thể dùng Pydantic để parse trực tiếp output thành kiểu dữ liệu [10]. Đối với đề tài, lựa chọn này phù hợp BR-02 và BR-04 vì backend có thể dùng cùng một contract để định nghĩa trường bắt buộc, kiểu dữ liệu và enum của Test Case [1].

JSON Schema Draft 2020-12 định nghĩa cơ chế mô tả cấu trúc tài liệu JSON; phần Core mô tả nền tảng của schema còn Validation Vocabulary định nghĩa các keyword để kiểm tra instance như type, properties, required và enum [11][12]. Tuy nhiên, Structured Outputs của nhà cung cấp có thể chỉ hỗ trợ một tập con JSON Schema; vì vậy schema dùng cho model cần giữ đơn giản, còn Pydantic ở backend tiếp tục đóng vai trò biên bảo vệ ứng dụng và phát hiện response bị refusal/incomplete hoặc lỗi ngoài contract [10].

```
{
 "type": "object",
 "properties": {
  "test_cases": {
   "type": "array",
   "items": {
    "type": "object",
    "properties": {
     "summary": {"type": "string"},
     "preconditions": {"type": "array", "items": {"type": "string"}},
     "steps": {"type": "array", "items": {"type": "string"}},
     "expected_result": {"type": "string"},
     "priority": {"type": "string", "enum": ["high", "medium", "low"]},
     "test_techniques": {"type": "array", "items": {"type": "string"}}
    },
    "required": ["summary", "preconditions", "steps", "expected_result", "priority",
"test_techniques"],
    "additionalProperties": false
   }
  }
 },
 "required": ["test_cases"],
 "additionalProperties": false
}
```

Mã 2.1: JSON Schema rút gọn cho đầu ra Test Case

Trường	Ý nghĩa	Ràng buộc
summary	Mô tả ngắn mục tiêu Test Case	Bắt buộc, string
preconditions	Điều kiện trước khi chạy	Mảng string
steps	Các bước thực hiện	Mảng string, ít nhất một bước ở tầng Pydantic
expected_result	Kết quả mong đợi	Bắt buộc, kiểm được
priority	Mức ưu tiên	high / medium / low
test_techniques	Kỹ thuật gợi ý	BVA / EP / Decision Table / State Transition / Other

Bảng 2.3: Cấu trúc Test Case chuẩn của hệ thống

Trong source của đề tài, schema này đóng vai trò contract giữa AI provider và tầng nghiệp vụ; response không thỏa contract không được lưu thành Test Case DRAFT hợp lệ.

2.2.4. Prompt engineering

Prompt của hệ thống không nên chỉ ghi "hãy sinh Test Case". Nó phải nêu vai trò, phạm vi Requirement, quy tắc BR cần tuân thủ, kỹ thuật kiểm thử cần cân nhắc, yêu cầu không bịa dữ liệu ngoài đặc tả và cách xử lý Requirement mơ hồ. Hướng dẫn prompting của OpenAI nhấn mạnh việc quản lý prompt như một thành phần có thể version hóa và đánh giá; đề tài ánh xạ điều này vào NC-09 - cấu hình prompt và mẫu sinh Test Case [13][1].

```
SYSTEM:
Bạn là Senior QA Engineer. Chỉ sinh Test Case dựa trên Requirement được cung cấp.
- Bao phủ happy path, negative case và boundary/EP khi có miền dữ liệu.
- Không tự bịa business rule ngoài Requirement. Nếu thông tin không đủ cơ sở, chỉ sinh các trường hợp có thể suy ra từ Requirement và để phần chưa rõ cho QA xử lý ở bước review.
- Mỗi Test Case phải thỏa schema do ứng dụng truyền vào.

USER:
Requirement ID: {requirement_id}
Module: {module_name}
Requirement text: {Requirement_text}
Acceptance criteria: {acceptance_criteria}
```

Mã 2.2: Prompt template minh họa

2.2.5. Human-in-the-loop và vòng đời Test Case

Trong hệ thống đề xuất, đầu ra do AI sinh không được xem là Test Case hợp lệ để sử dụng ngay. Mọi Test Case AI sinh được lưu ban đầu ở trạng thái DRAFT và không được chuyển trực tiếp sang APPROVED. Test Case cần đi qua trạng thái IN_REVIEW trước khi người dùng có quyền quyết định APPROVED hoặc NEEDS_FIX; trường hợp không phù hợp có thể chuyển sang REJECTED. Sau khi export thành công, Test Case chuyển sang EXPORTED. Khi Requirement nguồn thay đổi, Test Case APPROVED hoặc EXPORTED liên quan được chuyển về NEEDS_FIX để yêu cầu rà soát lại theo BR-08. Cách tổ chức này phù hợp với nguyên tắc human-in-the-loop: AI hỗ trợ tạo nội dung, còn quyết định sử dụng cuối cùng thuộc về con người [1][14]. Việc tách rõ trạng thái cũng giúp hệ thống ghi nhận phiên bản và audit log cho các thao tác sinh, sửa, duyệt và xuất Test Case.

Trạng thái	Ai tạo/chuyển	Ý nghĩa	Chuyển hợp lệ tiếp theo
DRAFT	AI/System	Bản nháp vừa sinh	IN_REVIEW / REJECTED
IN_REVIEW	QA	Đang rà soát/chỉnh sửa	NEEDS_FIX / APPROVED / REJECTED
NEEDS_FIX	QA/Manager	Cần chỉnh sửa thêm	IN_REVIEW
APPROVED	QA/Manager có quyền	Đã duyệt để sử dụng	EXPORTED / NEEDS_FIX nếu Requirement đổi
EXPORTED	System	Đã được xuất ra file	NEEDS_FIX khi Requirement đổi
REJECTED	QA/Manager	Loại bỏ khỏi bộ sử dụng	Có thể tạo Test Case mới, không tái dùng bản bị loại

Bảng 2.4: Vòng đời trạng thái Test Case trong mô hình HITL

2.2.6. Message Queue và xử lý tác vụ nền

Message Queue là cơ chế trung gian cho phép thành phần gửi yêu cầu công việc và thành phần xử lý nhận công việc theo cách tách rời về thời gian. Đối với các tác vụ có thời gian xử lý không ổn định như gọi API mô hình ngôn ngữ, việc xử lý đồng bộ trong HTTP request có thể khiến người dùng phải chờ lâu hoặc request bị timeout. Mô hình background job cho phép API tiếp nhận yêu cầu, tạo một bản ghi công việc, đưa task vào hàng đợi và trả kết quả tiếp nhận trước; worker sau đó xử lý công việc độc lập và cập nhật trạng thái.

Trong đề tài, RabbitMQ được sử dụng làm message broker và Celery đảm nhiệm background worker cho luồng sinh Test Case. Khi QA gửi yêu cầu sinh Test Case, backend tạo Generation Job và trả HTTP 202; task được đưa qua RabbitMQ để Celery worker gọi dịch vụ AI và cập nhật trạng thái theo vòng đời *QUEUED* → *RUNNING* → *COMPLETED* hoặc *FAILED*. Frontend có thể truy vấn trạng thái job thay vì giữ một HTTP request mở trong toàn bộ thời gian mô hình xử lý. Cách tổ chức này đáp

ứng yêu cầu xử lý bất đồng bộ cho chức năng AI và tách phần web API khỏi tác vụ có độ trễ lớn.

2.2.7. Vector Embedding, Cosine Similarity và pgvector

Vector Embedding biểu diễn nội dung văn bản dưới dạng một vector số trong không gian nhiều chiều, trong đó các nội dung có ý nghĩa tương tự có xu hướng nằm gần nhau. Nhờ đó, hai Test Case không cần giống nhau hoàn toàn về câu chữ vẫn có thể được phát hiện là tương tự về ngữ nghĩa.

Cosine similarity đánh giá mức độ tương đồng giữa hai vector thông qua góc giữa chúng. Trong đề tài, embedding của Test Case được tạo bằng mô hình text-embedding-3-small với 1536 chiều và lưu trong PostgreSQL thông qua extension pgvector. Hệ thống sử dụng cosine similarity và ngưỡng mặc định 0,85 để tìm các Test Case có khả năng trùng lặp; HNSW index với vector_cosine_ops hỗ trợ truy vấn vector hiệu quả hơn khi số lượng dữ liệu tăng. Kết quả similarity chỉ được dùng làm cảnh báo/candidate cho người dùng rà soát, không tự động quyết định hai Test Case phải được gộp. Cách xử lý này phù hợp với NC-05 và nguyên tắc human-in-the-loop của hệ thống.

2.2.8. Khung đánh giá chất lượng và hiệu quả hệ thống

Vì đề tài không tự động thực thi Test Case được sinh trên source code của một hệ thống đích, việc đánh giá tập trung vào chất lượng và khả năng sử dụng của đầu ra AI. Các chỉ số gồm Schema Validity, Human Acceptance Rate, Requirement Traceability, Boundary Coverage, Duplicate Rate và Time Saving [15].

Bảng 2.5 định nghĩa cách đo và nguồn dữ liệu cho từng chỉ số; kết quả đo thực tế được trình bày tại Chương 5. Chỉ số chưa thu thập đủ dữ liệu sẽ được ghi rõ là chưa đo được.

Metric	Công thức	Nguồn dữ liệu	Liên hệ BRD
Schema validity rate	Valid structured response / completed non-refusal response	Generation log	BR-02, BR-04
Human acceptance rate	Số Test Case AI được chấp nhận sau review với tối đa chỉnh sửa nhỏ / tổng Test Case AI được review nhân cho 100%	Audit log + version diff + kết quả QA review	BR-01
Requirement traceability	TC có Requirement nguồn / tổng TC	DB + RTM	BR-03
Boundary Coverage	Boundary có TC / boundary trong gold set	Gold set + QA review	NC-03, BR-09
Duplicate rate	TC bị cảnh báo/gộp / tổng TC	Duplicate service log	NC-05
Time saving	So sánh thời gian thủ công và có AI	Benchmark	KPI BRD

Bảng 2.5: Khung chỉ số đánh giá chất lượng và hiệu quả hệ thống

CHƯƠNG 3: PHÂN TÍCH & THIẾT KẾ HỆ THỐNG

Chương 3 chuyển hóa các yêu cầu trong BRD thành đặc tả phần mềm và thiết kế hệ thống. Các yêu cầu NC-01 đến NC-12 và BR-01 đến BR-09 được sử dụng làm nguồn truy vết cho yêu cầu chức năng, Use Case, mô hình dữ liệu, kiến trúc và Test Case. Thiết kế trong chương này được cập nhật theo kiến trúc và mô hình dữ liệu thực tế của phiên bản hoàn thiện. [1]

Chương này trình bày quá trình chuyển hóa yêu cầu trong BRD thành mô hình phân tích và thiết kế của hệ thống. Nội dung được tổ chức theo chuỗi truy vết từ yêu cầu chức năng, quy tắc nghiệp vụ và yêu cầu phi chức năng đến Use Case, luồng hoạt

động, tương tác tuần tự, mô hình dữ liệu, mô hình lớp, giao diện và kiến trúc tổng thể. Ba nhóm người dùng chính gồm Kỹ sư QA, Quản lý kiểm thử và Quản trị hệ thống; OpenAI API đóng vai trò hệ thống ngoài hỗ trợ chức năng sinh Test Case và tạo embedding.

Trọng tâm thiết kế là bảo đảm đầu ra AI không được sử dụng trực tiếp mà phải đi qua quy trình human-in-the-loop; mỗi Test Case được truy vết về Requirement và Module nguồn; các thay đổi quan trọng được lưu phiên bản và audit; các tác vụ AI có độ trễ được xử lý bất đồng bộ. Các mô hình trong chương được xây dựng thống nhất với cấu trúc triển khai thực tế trình bày ở Chương 4.

3.1. Đặc tả yêu cầu chức năng (SRS)

Bảng 3.1 tổng hợp các yêu cầu chức năng được chuyển hóa từ phạm vi NC trong BRD và dùng làm đầu mối truy vết cho Use Case và kiểm thử.

Mã YC	Tên chức năng	Mô tả chi tiết	Actor khởi tạo/chính	NC liên quan
YC-01	Nhập đặc tả yêu cầu	Tạo Requirement từ nội dung SRS/AC; gắn module; validate độ dài và quyền record.	Kỹ sư QA	NC-01
YC-02	Sinh Test Case tự động	QA gửi yêu cầu sinh Test Case cho Requirement do mình sở hữu; hệ thống tạo Generation Job, xử lý tác vụ nền qua RabbitMQ/Celery, gọi OpenAI API bằng Structured Outputs và lưu Test Case hợp lệ ở trạng thái DRAFT; hỗ trợ gợi ý các trường hợp boundary/EP khi phù hợp.	Kỹ sư QA; OpenAI API là hệ ngoài	NC-02, NC-03
YC-03	Rà soát và tối ưu	Hiển thị bản AI, cho edit/diff, duplicate warning/merge, đánh dấu review; không tự động coi AI output là hợp lệ.	Kỹ sư QA	NC-04, NC-05

Mã YC	Tên chức năng	Mô tả chi tiết	Actor khởi tạo/chính	NC liên quan
YC-04	Tổ chức, duyệt và xuất bản	Nhóm theo module/tag, theo dõi Coverage, duyệt theo quyền, export chỉ các Test Case APPROVED.	Kỹ sư QA/Quản lý kiểm thử	NC-06, NC-07, NC-12
YC-05	Quản lý hệ thống và phiên bản	Auth/RBAC, user management, prompt config, version history, audit log; Requirement đổi thì đánh dấu Test Case cần review.	Admin/Quản lý/System	NC-08, NC-09, NC-10, NC-11

Bảng 3.1: Đặc tả yêu cầu chức năng của hệ thống

3.2. Danh mục quy tắc nghiệp vụ

Mã BR	Tên	Mô tả hình thức hóa	Phân loại
BR-01	Bắt buộc human review	Mọi Test Case AI sinh phải được rà soát và duyệt trước khi đưa vào sử dụng; không sử dụng trực tiếp đầu ra chưa duyệt.	Bắt buộc
BR-02	Cấu trúc Test Case	Mỗi Test Case có tối thiểu mô tả, bước thực hiện, kết quả mong đợi, mức ưu tiên.	Bắt buộc
BR-03	Truy vết nguồn	Mỗi Test Case gắn một module/tính năng và tối thiểu một Requirement nguồn.	Bắt buộc
BR-04	Schema validation	Đầu ra AI phải tuân theo Structured Output schema. Nếu output không parse/validate được thì hệ thống thực hiện sinh lại có giới hạn; sau khi vượt số lần thử cho phép phải trả lỗi và không lưu output đó như Test Case hợp lệ.	Bắt buộc

Mã BR	Tên	Mô tả hình thức hóa	Phân loại
BR-05	Quyền duyệt/export	Chỉ người dùng có quyền mới được duyệt hoặc export.	Bắt buộc
BR-06	Version và audit	Mọi lần sinh lại, chỉnh sửa, duyệt phải được ghi lịch sử phiên bản và nhật ký; hệ thống cũng ghi generate/export để đáp ứng NC-11.	Bắt buộc
BR-07	Bảo mật dữ liệu	Requirement và Test Case là dữ liệu nội bộ, phải được phân quyền truy cập.	Bắt buộc
BR-08	Requirement thay đổi	Test Case liên quan phải được đánh dấu cần rà soát lại khi Requirement thay đổi.	Chính sách
BR-09	Chính sách chất lượng	Khuyến khích kiểm soát boundary Coverage và ưu tiên rà soát Test Case priority cao trước.	Chính sách

Bảng 3.2: Danh mục quy tắc nghiệp vụ BR-01 đến BR-09 [1]

Trong implementation hiện tại, số lần **retry do sai schema tối đa là 2 lần** ngoài lần gọi đầu tiên.

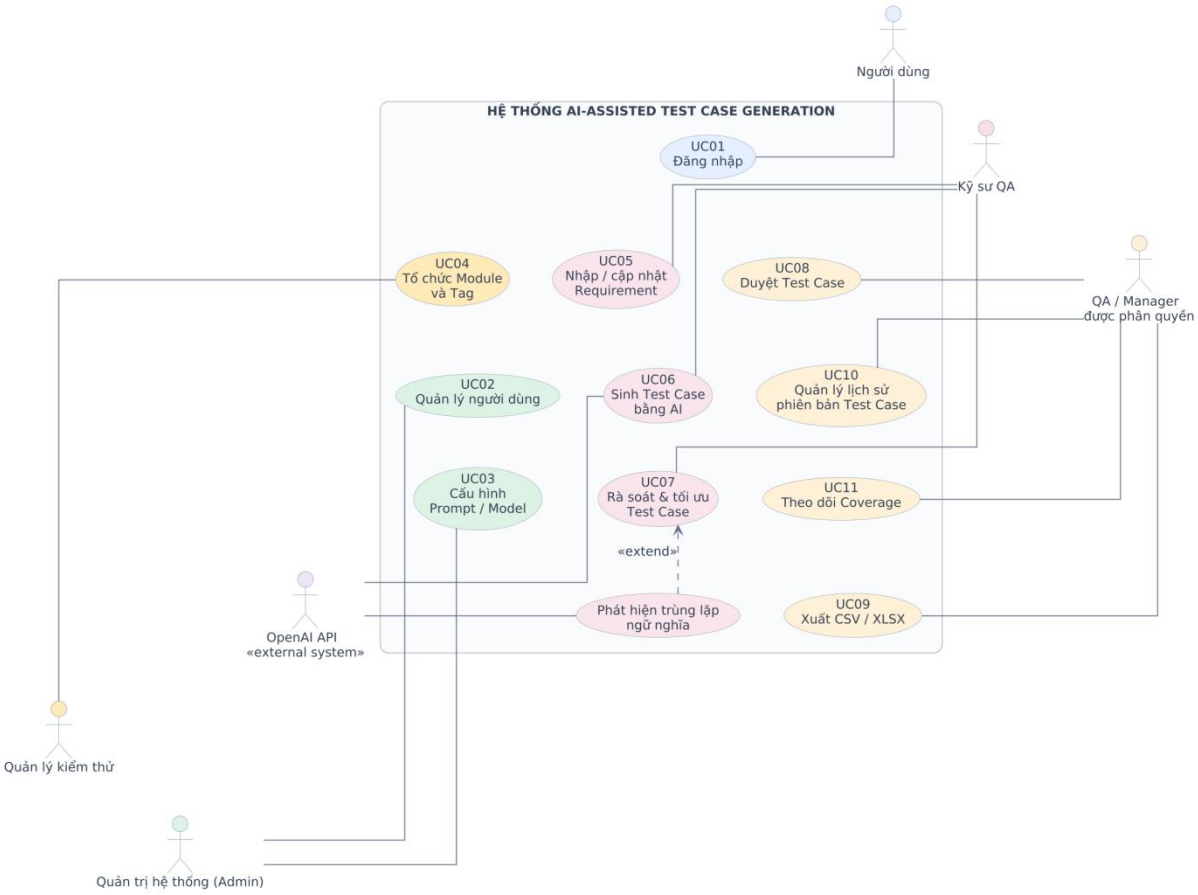
3.3. Đặc tả yêu cầu phi chức năng

Mã NFR	Yêu cầu	Tiêu chí kiểm được
NFR-01	Bảo mật secret	API key, JWT secret, DB URL chỉ đọc từ environment; không commit .env.
NFR-02	Xác thực/phân quyền	JWT; kiểm role và ownership/record ở server; endpoint trái quyền trả 403.
NFR-03	Tính đúng cấu trúc AI	Output dùng Structured Outputs; backend validate Pydantic; không lưu response không hợp lệ như DRAFT.
NFR-04	Khả năng truy vết	TestCase có Requirement/module; audit có user/action/record/time; version snapshot bất biến.
NFR-05	Độ tin cậy	API lỗi/rate-limit/refusal/incomplete có error code rõ; retry giới hạn và có log.

Mã NFR	Yêu cầu	Tiêu chí kiểm được
NFR-06	Hiệu năng/khả năng đáp ứng	Các danh sách được phân trang tại DB; tác vụ AI có thời gian xử lý dài được chạy dưới dạng background job qua RabbitMQ và Celery; API chỉ tiếp nhận tác vụ và cho phép client truy vấn trạng thái thay vì giữ HTTP request chờ mô hình hoàn thành.
NFR-07	Khả năng bảo trì	Backend tách Router → Service → Repository; OpenAI nằm adapter common/ai; model/config không hardcode.
NFR-08	Triển khai	Hệ thống có Alembic migration, seed demo, README hướng dẫn chạy lại, cấu hình qua environment và GitHub Actions CI. Phiên bản hiện tại được nghiệm thu trên môi trường local.

Bảng 3.3: Yêu cầu phi chức năng có tiêu chí kiểm được

3.4. Sơ đồ Use Case và đặc tả chi tiết



Hình 3.1: Sơ đồ Use Case

Hình 3.1 thể hiện bức tranh tổng quát về các chức năng của hệ thống và phạm vi thao tác của từng vai trò. Kỹ sư QA thực hiện các nghiệp vụ liên quan đến Requirement, sinh Test Case, rà soát và xử lý Test Case; Quản lý kiểm thử tham gia các nghiệp vụ quản lý Module, theo dõi Coverage, duyệt và export; Admin chịu trách nhiệm quản lý người dùng và cấu hình Prompt/Model.

Luồng nghiệp vụ cốt lõi tập trung ở UC05–UC10: Requirement được nhập làm dữ liệu nguồn, AI sinh Test Case ở trạng thái DRAFT, QA rà soát/chỉnh sửa và xử lý duplicate, người có quyền quyết định duyệt, sau đó Test Case APPROVED mới được export. Lịch sử phiên bản và audit được sử dụng xuyên suốt để bảo đảm khả năng truy vết.

UC	Tên Use Case	Actor chính	YC/BR nổi bật
UC01	Đăng nhập	Người dùng	YC-05, BR-07
UC02	Quản lý người dùng	Admin	YC-05, BR-05/07

UC	Tên Use Case	Actor chính	YC/BR nổi bật
UC03	Cấu hình Prompt/Model	Admin	YC-05, NC-09
UC04	Tổ chức module và tag	Quản lý kiểm thử	YC-04, BR-03
UC05	Nhập đặc tả yêu cầu	Kỹ sư QA	YC-01, BR-07
UC06	Sinh Test Case tự động	Kỹ sư QA	YC-02, BR-02/03/04/06
UC07	Rà soát và tối ưu	Kỹ sư QA	YC-03, BR-01/06/09
UC08	Duyệt Test Case	QA/Manager có quyền	YC-04, BR-01/05/06
UC09	Xuất CSV/Excel	QA/Manager có quyền	YC-04, BR-01/05/06
UC10	Quản lý lịch sử phiên bản Test Case	QA/Manager có quyền	YC-05, BR-01/05/06
UC11	Theo dõi Coverage	QA/Manager có quyền	YC-04, BR-01/05/06

Bảng 3.4: Danh mục Use Case chính

3.4.1. Đặc tả Use Case "Đăng nhập"

Thuộc tính	Nội dung
Use Case ID	UC01
Tên Use Case	Đăng nhập
Actor chính/phụ	Người dùng (QA/Manager/Admin)
Mục tiêu	Xác thực người dùng và cấp token theo vai trò.
Tiền điều kiện	Tài khoản tồn tại và chưa bị vô hiệu hóa.
Hậu điều kiện	Trả access token; không trả password hash.

Thuộc tính	Nội dung
Luồng chính	1. Người dùng nhập email và mật khẩu. 2. Hệ thống validate input. 3. Hệ thống lấy user qua repository và kiểm password hash. 4. Nếu hợp lệ, hệ thống phát JWT có thời hạn và trả User DTO.
Luồng thay thế/ngoại lệ	- Sai thông tin → 401; không tiết lộ email hay password sai cụ thể. - Tài khoản inactive → 403. - Tài khoản đang bị khóa do đăng nhập sai nhiều lần → 429.

3.4.2. Đặc tả Use Case "Quản lý người dùng"

Thuộc tính	Nội dung
Use Case ID	UC02
Tên Use Case	Quản lý người dùng
Actor chính/phụ	Admin
Mục tiêu	Tạo/cập nhật/vô hiệu hóa tài khoản và role.
Tiền điều kiện	Admin đăng nhập và có quyền quản trị.
Hậu điều kiện	User được cập nhật; audit log ghi thao tác.
Luồng chính	1. Admin mở danh sách user. 2. Tạo hoặc sửa user qua form có validation. 3. Service kiểm email unique, role hợp lệ. 4. Repository ghi DB; audit ghi before/after.
Luồng thay thế/ngoại lệ	- Email trùng → 409. - Role không hợp lệ → 422. - User không phải Admin → 403.

3.4.3. Đặc tả Use Case "Cấu hình Prompt/Model"

Thuộc tính	Nội dung
Use Case ID	UC03
Tên Use Case	Cấu hình Prompt/Model
Actor chính/phụ	Admin
Mục tiêu	Quản lý prompt template, schema version và model cấu hình.
Tiền điều kiện	Admin đã đăng nhập.
Hậu điều kiện	Phiên bản cấu hình mới được lưu; không sửa xóa history cũ.
Luồng chính	1. Admin mở cấu hình AI. 2. Nhập prompt template/model/schema metadata. 3. Service validate cấu hình. 4. Lưu phiên bản mới và đánh dấu active. 5. Audit ghi người thay đổi.
Luồng thay thế/ngoại lệ	- Thiếu {Requirement_text} hoặc {acceptance_criteria} trong user_prompt_template → 422. - Prompt chứa placeholder không được hỗ trợ → 422. - Không phải Admin → 403.

3.4.4. Đặc tả Use Case "Tổ chức module và tag"

Thuộc tính	Nội dung
Use Case ID	UC04
Tên Use Case	Tổ chức module
Actor chính/phụ	Quản lý kiểm thử
Mục tiêu	Tạo/sửa module và nhóm Requirement/Test Case để truy vết.
Tiền điều kiện	Quản lý kiểm thử đã đăng nhập và có quyền quản lý module.
Hậu điều kiện	Module tồn tại; dữ liệu liên quan truy vấn được theo module.

Thuộc tính	Nội dung
Luồng chính	1. Mở chức năng quản lý module. 2. Tạo module hoặc module con. 3. Service kiểm tên và parent hợp lệ. 4. Repository lưu và trả DTO.
Luồng thay thế/ngoại lệ	- parent_id không tồn tại → 404. - Tạo vòng lặp cây module → 422.

3.4.5. Đặc tả Use Case "Nhập đặc tả yêu cầu"

Thuộc tính	Nội dung
Use Case ID	UC05
Tên Use Case	Nhập đặc tả yêu cầu
Actor chính/phụ	Kỹ sư QA
Mục tiêu	Lưu Requirement/AC làm nguồn sinh Test Case.
Tiền điều kiện	QA đăng nhập và có quyền trên module.
Hậu điều kiện	Requirement được lưu với id, module_id, created_by, lock_version; audit create được ghi.
Luồng chính	1. QA chọn module. 2. Nhập Requirement text và acceptance criteria. 3. Pydantic validate input. 4. Service kiểm record permission. 5. Repository lưu Requirement với lock_version = 1; Audit Log ghi thao tác tạo.
Luồng thay thế/ngoại lệ	- Text/input không hợp lệ → 422. - Module không tồn tại → 404. - QA sửa Requirement không do mình tạo → 403. - lock_version cũ khi cập nhật → 409.

3.4.6. Đặc tả Use Case "Sinh Test Case tự động"

Thuộc tính	Nội dung
Use Case ID	UC06
Tên Use Case	Sinh Test Case tự động
Actor chính/phụ	Kỹ sư QA; actor phụ: OpenAI API
Mục tiêu	Sinh danh sách Test Case nháp có cấu trúc từ Requirement.
Tiền điều kiện	Requirement tồn tại, QA có quyền; prompt config active.
Hậu điều kiện	Test Case hợp lệ được lưu DRAFT, gắn Requirement/module, version và audit generate.
Luồng chính	1. QA chọn Requirement và yêu cầu sinh Test Case. 2. Backend kiểm Requirement tồn tại, QA sở hữu Requirement và kiểm rate limit. 3. Hệ thống tạo GenerationJob ở trạng thái QUEUED. 4. Task được chuyển vào RabbitMQ để Celery worker xử lý. 5. Worker chuyển Job sang RUNNING, tải Requirement và PromptConfig đang active. 6. OpenAIAdapter gọi OpenAI bằng Structured Outputs/Pydantic schema. 7. Nếu output sai schema, adapter thử lại tối đa 2 lần. 8. Output hợp lệ được lưu thành Test Case DRAFT, gắn requirement_id, module_id, tạo version và audit. 9. Job chuyển sang COMPLETED; nếu lỗi chuyển sang FAILED. 10. Frontend truy vấn trạng thái Job và tải Test Case khi hoàn tất.
Luồng thay thế/ngoại lệ	- OpenAI timeout/rate limit → trả lỗi dịch vụ, không tạo Test Case rỗng. - Refusal/incomplete/parse failure → không lưu như output hợp lệ; retry giới hạn nếu cấu hình cho phép. - Requirement mơ hồ → tạo review_note thay vì bịa rule.

3.4.7. Đặc tả Use Case "Rà soát và tối ưu Test Case"

Thuộc tính	Nội dung
Use Case ID	UC07
Tên Use Case	Rà soát và tối ưu Test Case
Actor chính/phụ	Kỹ sư QA
Mục tiêu	Cho QA sửa nội dung, review diff và xử lý duplicate trước khi duyệt.
Tiền điều kiện	Test Case ở DRAFT/IN_REVIEW/NEEDS_FIX, QA có quyền record.
Hậu điều kiện	Nội dung mới được lưu thành version; audit edit/merge; trạng thái phù hợp.
Luồng chính	1. QA mở danh sách cần rà soát. 2. Hệ thống hiển thị AI Original và Current Version. 3. QA chỉnh sửa hoặc chọn cặp duplicate. 4. Service validate update và record permission. 5. Repository lưu snapshot version mới. 6. Nếu người dùng xác nhận duplicate và similarity vẫn đạt ngưỡng, hệ thống giữ Test Case đích làm canonical; Test Case nguồn chuyển sang REJECTED; cập nhật review note, lock version, version snapshot và audit cho cả hai bản ghi.
Luồng thay thế/ngoại lệ	- Similarity chỉ là cảnh báo; QA có thể không merge. - Xung đột optimistic locking, yêu cầu tải lại → 409.

3.4.8. Đặc tả Use Case "Duyệt Test Case"

Thuộc tính	Nội dung
Use Case ID	UC08
Tên Use Case	Duyệt Test Case
Actor chính/phụ	QA/Manager được phân quyền
Mục tiêu	Chuyển Test Case đã review sang APPROVED theo BR-01/BR-05.

Thuộc tính	Nội dung
Tiền điều kiện	Test Case đang ở trạng thái IN_REVIEW, đủ các trường bắt buộc và user là QA/Manager có quyền trên record.
Hậu điều kiện	Status=APPROVED; version/audit approval được ghi.
Luồng chính	1. User chọn Test Case đã review. 2. Service kiểm status transition và quyền. 3. Service kiểm các trường BR-02 và traceability BR-03. 4. Repository cập nhật status trong transaction. 5. Audit ghi actor, time, record và before/after.
Luồng thay thế/ngoại lệ	- DRAFT → APPROVED trực tiếp → 409. - Thiếu quyền → 403. - Thiếu field bắt buộc → 422.

3.4.9. Đặc tả Use Case "Xuất file CSV/Excel"

Thuộc tính	Nội dung
Use Case ID	UC09
Tên Use Case	Xuất file CSV/Excel
Actor chính/phụ	QA/Manager được phân quyền
Mục tiêu	Xuất các Test Case APPROVED theo module/template.
Tiền điều kiện	Có ít nhất một APPROVED Test Case và user có quyền export.
Hậu điều kiện	File CSV/XLSX được tạo thành công; các Test Case được export chuyển từ APPROVED sang EXPORTED; Audit Log ghi sự kiện export.
Luồng chính	1. User chọn module và định dạng. 2. Service query các Test Case APPROVED bằng pagination/filter DB. 3. Map DTO sang template CSV/XLSX. 4. Trả file cho client. 5. Audit ghi export event.

Thuộc tính	Nội dung
Luồng thay thế/ngoại lệ	- Không có Test Case APPROVED → 422. - DRAFT/IN_REVIEW/NEEDS_FIX/REJECTED không được export. - User không thuộc QA/Manager → 403.

3.4.10. Đặc tả Use Case "Quản lý lịch sử phiên bản Test Case"

Thuộc tính	Nội dung
Use Case ID	UC10
Tên Use Case	Quản lý lịch sử phiên bản Test Case
Actor chính/phụ	QA/Manager được phân quyền
Mục tiêu	Xem lịch sử các phiên bản của Test Case, so sánh nội dung giữa các phiên bản và khôi phục một phiên bản trước đó khi cần.
Tiền điều kiện	Test Case đã tồn tại trong hệ thống và người dùng có quyền truy cập.
Hậu điều kiện	Nếu chỉ xem/so sánh, dữ liệu Test Case không thay đổi. Nếu khôi phục, nội dung của phiên bản được chọn trở thành trạng thái hiện tại; hệ thống tạo một phiên bản mới, tăng lock_version và ghi AuditLog để bảo toàn lịch sử.
Luồng chính	1. User mở một Test Case. 2. Hệ thống tải danh sách TestCaseVersion theo thứ tự phiên bản. 3. User chọn một phiên bản để xem chi tiết hoặc chọn hai phiên bản để so sánh. 4. Hệ thống hiển thị snapshot và các khác biệt giữa các phiên bản. 5. Nếu cần, User chọn Restore tại một phiên bản trước. 6. Service kiểm tra quyền và lock_version hiện tại. 7. Hệ thống khôi phục dữ liệu từ snapshot được chọn vào Test Case hiện tại. 8. Hệ thống tạo TestCaseVersion mới, tăng lock_version và ghi AuditLog cho thao tác restore.

Thuộc tính	Nội dung
Luồng thay thế/ngoại lệ	- Test Case hoặc phiên bản không tồn tại → 404. - User không có quyền thao tác → 403. - lock_version không còn khớp do Test Case đã được cập nhật bởi phiên khác → 409. - Dữ liệu restore không hợp lệ → hệ thống từ chối cập nhật và không tạo phiên bản mới.

3.4.11. Đặc tả Use Case "Theo dõi Coverage"

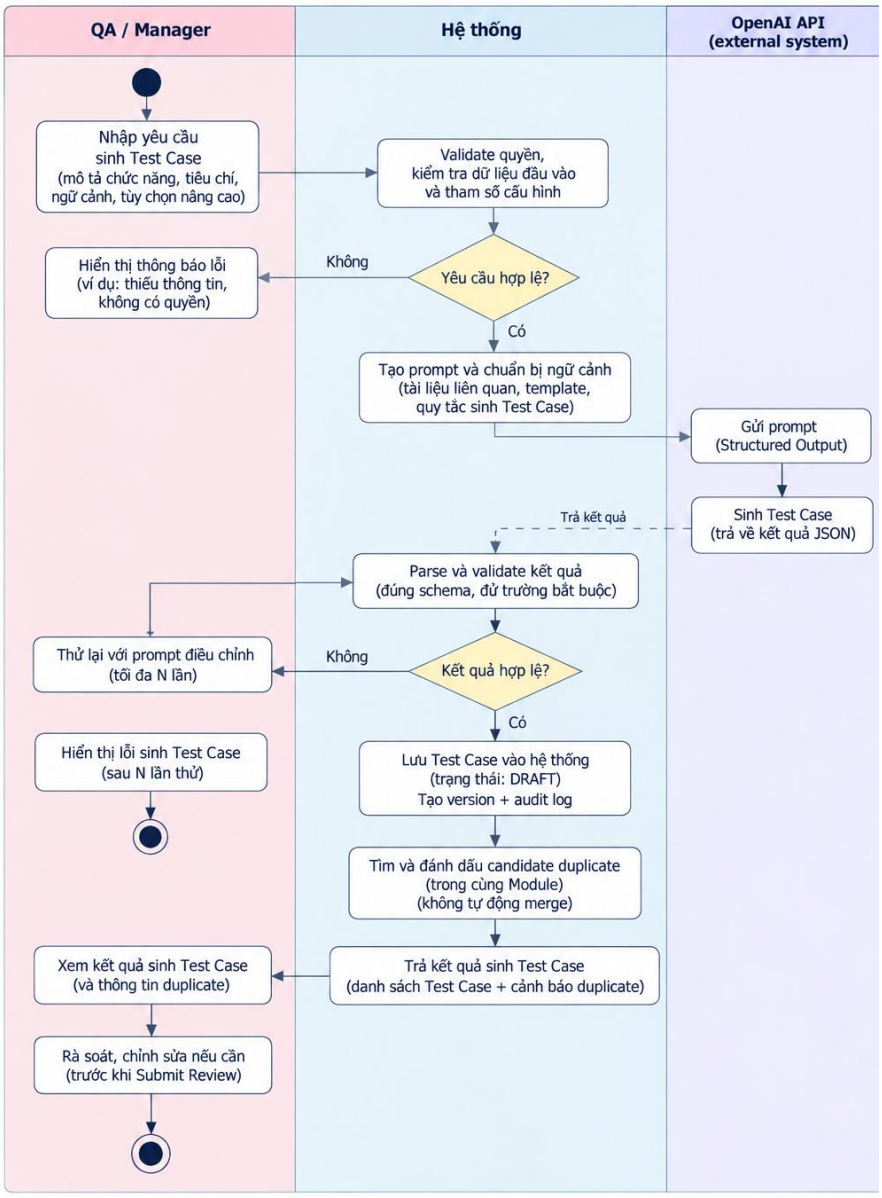
Thuộc tính	Nội dung
Use Case ID	UC11
Tên Use Case	Theo dõi Coverage
Actor chính/phụ	QA/Manager được phân quyền
Mục tiêu	Theo dõi số lượng Requirement, Test Case và mức độ bao phủ Requirement theo từng Module để hỗ trợ QA/Manager đánh giá phạm vi kiểm thử.
Tiền điều kiện	User đã đăng nhập, có quyền truy cập Module cần xem và Module tồn tại trong hệ thống.
Hậu điều kiện	Hệ thống hiển thị số liệu Coverage hiện tại của Module; dữ liệu Requirement/Test Case không bị thay đổi.
Luồng chính	1. User chọn module cần theo dõi. 2. Frontend gửi module_id đến backend. 3. Hệ thống kiểm tra Module và quyền truy cập của user. 4. Service tổng hợp số Requirement và các Test Case liên kết với Module/Requirement. 5. Hệ thống tính mức độ bao phủ Requirement của Module từ dữ liệu hiện có. 6. Backend trả số liệu Coverage cho frontend. 7. Frontend hiển thị số lượng Requirement, Test Case và tỷ lệ Coverage để user theo dõi.
Luồng thay thế/ngoại lệ	- Module không tồn tại → 404. - User không có quyền truy cập Module → 403. - Module chưa có dữ liệu phù hợp để thống kê → hệ thống hiển thị Coverage tương ứng với dữ liệu hiện có, không phát sinh thay đổi dữ liệu.

3.5. Thiết kế luồng nghiệp vụ cốt lõi

3.5.1. Sơ đồ hoạt động

3.5.1.1. Sinh Test Case bằng AI

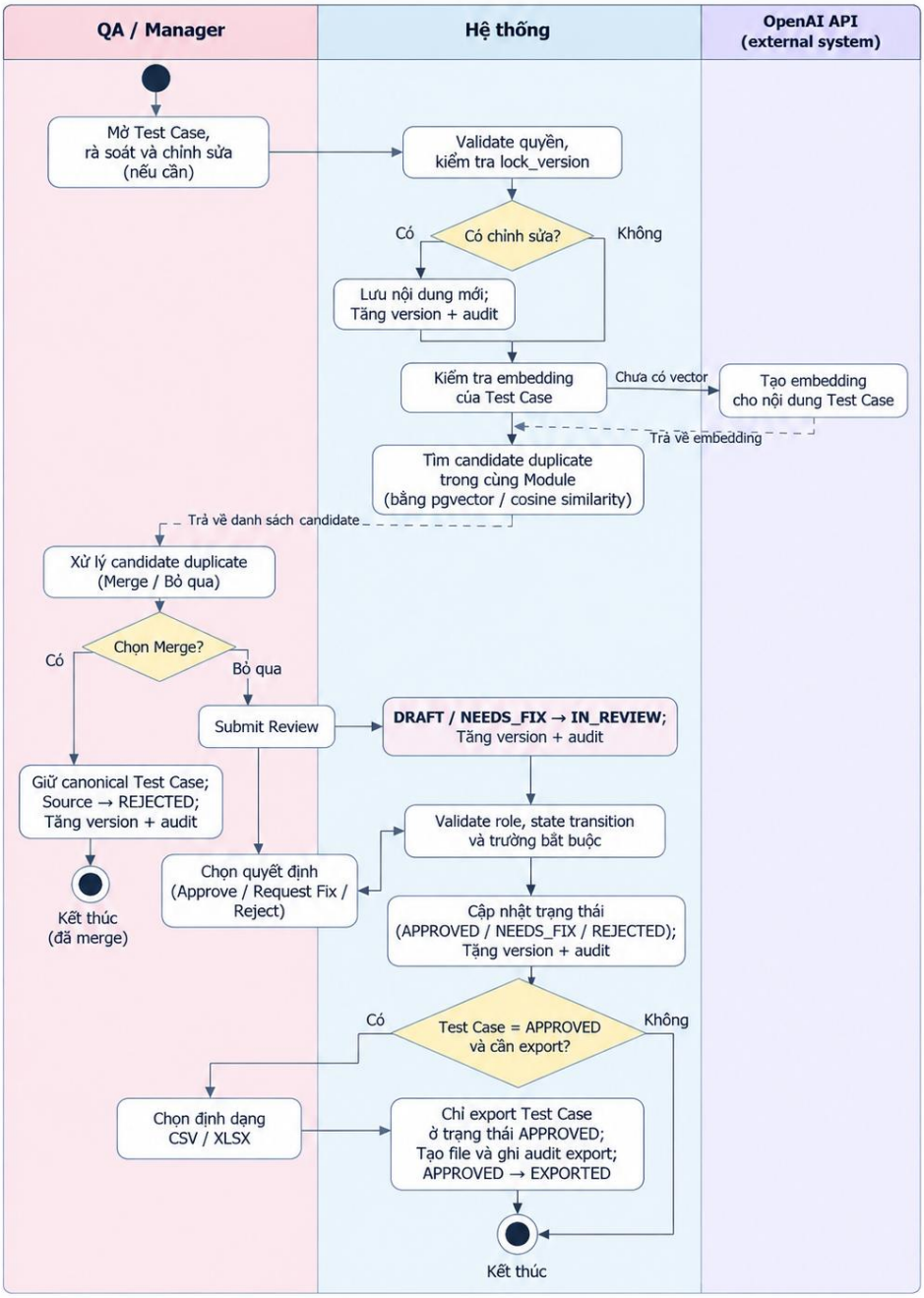
Luồng sinh Test Case bằng AI là nghiệp vụ trung tâm của hệ thống và hiện thực UC06. Hình 3.2 mô tả quá trình từ khi QA lựa chọn Requirement, hệ thống kiểm tra quyền và dữ liệu đầu vào, tiếp nhận yêu cầu sinh Test Case cho đến khi kết quả hợp lệ được lưu ở trạng thái DRAFT để tiếp tục bước rà soát. Nếu việc sinh hoặc kiểm tra đầu ra thất bại, hệ thống không lưu dữ liệu không hợp lệ thành Test Case.



Hình 3.2: Sơ đồ hoạt động "Sinh Test Case bằng AI"

Điểm quan trọng của luồng này là AI chỉ tạo bản nháp. Structured Output và validation bảo vệ cấu trúc dữ liệu, còn quyết định sử dụng Test Case vẫn thuộc về con người theo BR-01 và BR-04.

3.5.1.2. Rà soát, phát hiện trùng lặp, duyệt và export Test Case

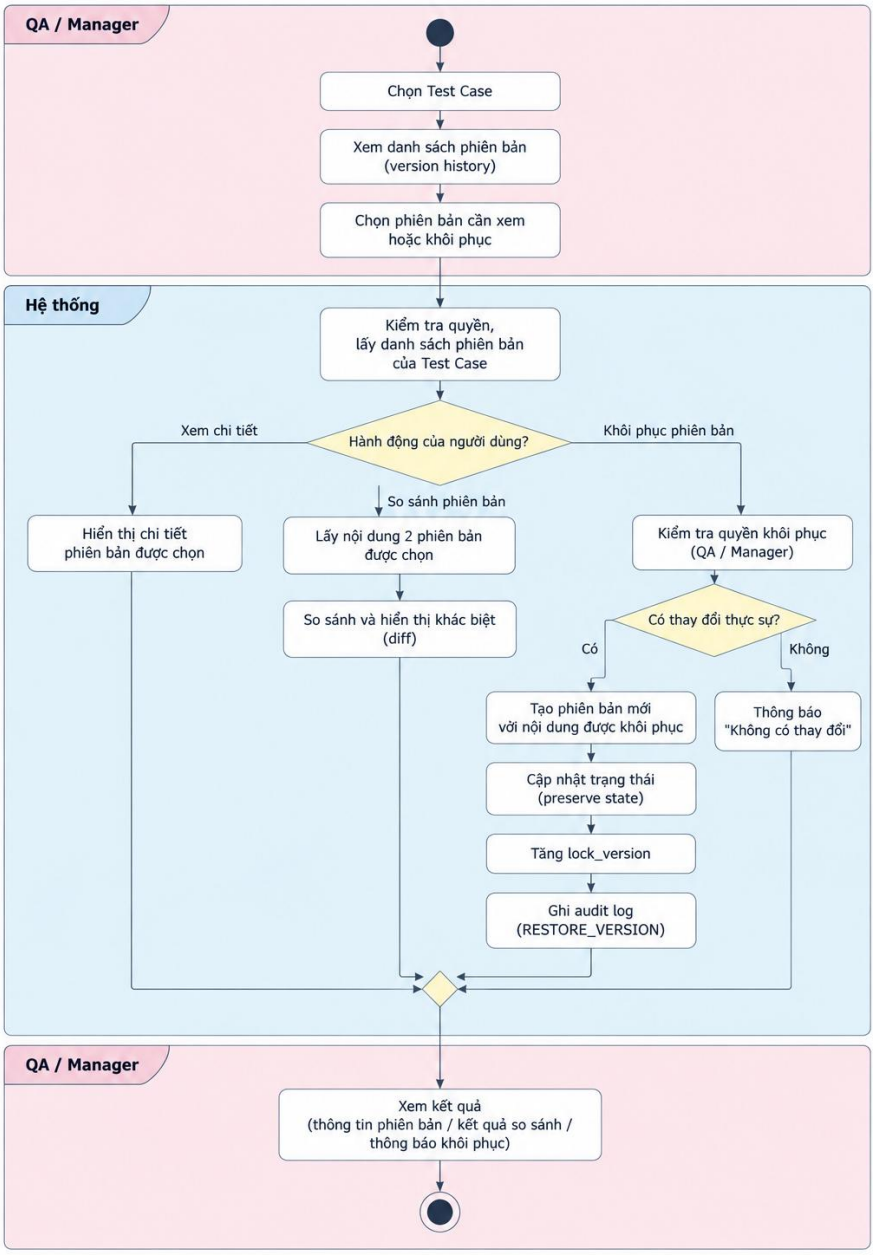


Hình 3.3: Sơ đồ hoạt động "Rà soát, khử trùng lặp, duyệt và export Test Case"

Phát hiện duplicate chỉ cung cấp candidate dựa trên mức độ tương đồng, hệ thống không tự động merge. Quyết định cuối cùng vẫn thuộc QA nhằm bảo đảm nguyên tắc human-in-the-loop của BRD.

3.5.1.3. Cập nhật Requirement theo BR-08

Hình 3.4 mô tả chính sách BR-08 khi Requirement nguồn được cập nhật. Hệ thống dùng optimistic locking để kiểm soát cập nhật đồng thời. Sau một yêu cầu cập nhật hợp lệ, các Test Case liên quan đang ở APPROVED hoặc EXPORTED được chuyển sang NEEDS_FIX, tăng lock_version, tạo snapshot phiên bản và ghi audit để buộc rà soát lại trước khi tiếp tục sử dụng.



Hình 3.4: Sơ đồ hoạt động "Update Requirement & đánh dấu Test Case cần rà soát lại"

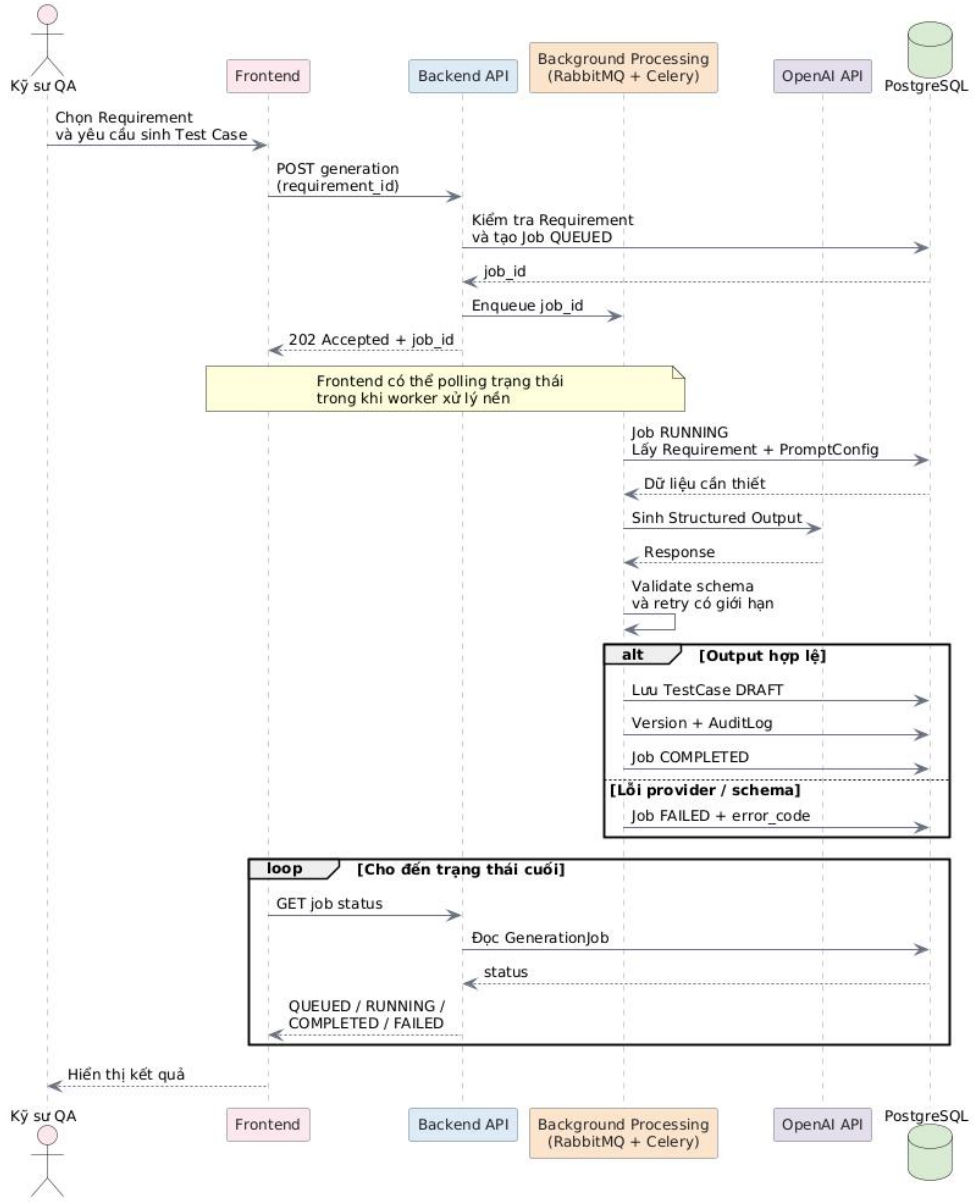
Cách xử lý bảo thủ này ưu tiên tính an toàn: mọi cập nhật hợp lệ của Requirement đều làm các Test Case đã APPROVED/EXPORTED liên quan quay về

NEEDS_FIX. Nhờ đó hệ thống không tiếp tục sử dụng một Test Case đã duyệt mà chưa được rà soát lại sau khi Requirement nguồn được chỉnh sửa.

3.5.2. Sơ đồ tuần tự

3.5.2.1. Sinh Test Case bất đồng bộ bằng AI

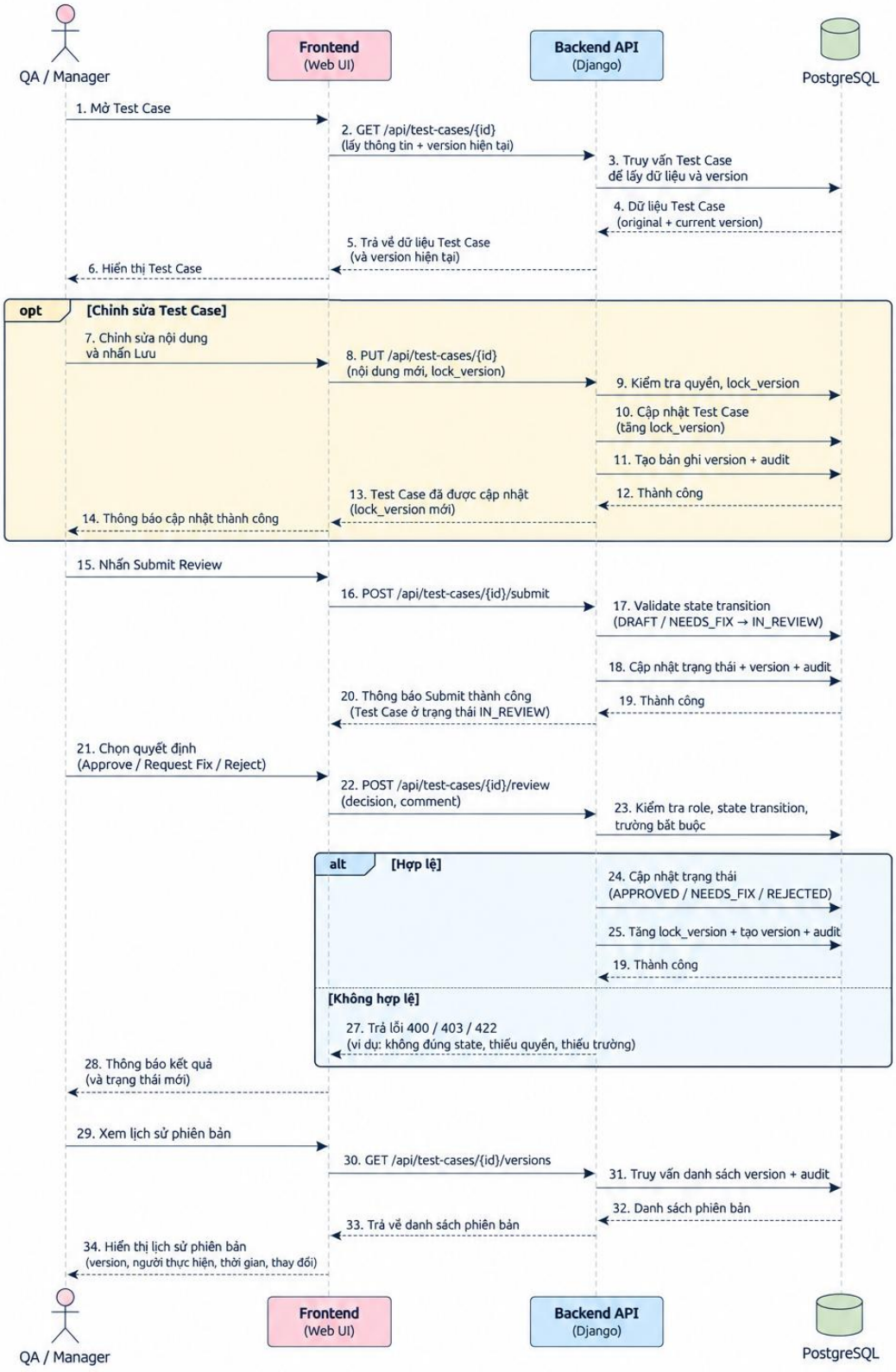
Sơ đồ tuần tự ở Hình 3.5 làm rõ tương tác theo thời gian của UC06 giữa giao diện, backend, tác vụ nền, dịch vụ AI và tầng dữ liệu. Backend không giữ HTTP request trong toàn bộ thời gian mô hình xử lý mà tạo Generation Job và trả trạng thái tiếp nhận cho frontend. Worker xử lý tác vụ nền, gọi dịch vụ AI và chỉ lưu Test Case khi kết quả vượt qua bước validation. Frontend theo dõi trạng thái Job để lấy kết quả khi quá trình hoàn tất.



Hình 3.5: Sơ đồ tuần tự "Sinh Test Case bất đồng bộ bằng AI"

3.5.2.2. Rà soát và duyệt Test Case

Hình 3.6 mô tả tương tác của quá trình rà soát và duyệt Test Case. Trong implementation, frontend tải Test Case hiện tại và lịch sử version bằng các request riêng; khi cập nhật, backend kiểm tra quyền, trạng thái hiện tại và lock_version. Mỗi thay đổi nghiệp vụ quan trọng tạo snapshot TestCaseVersion và AuditLog. Quy tắc trạng thái ngăn Test Case DRAFT được chuyển trực tiếp sang APPROVED.

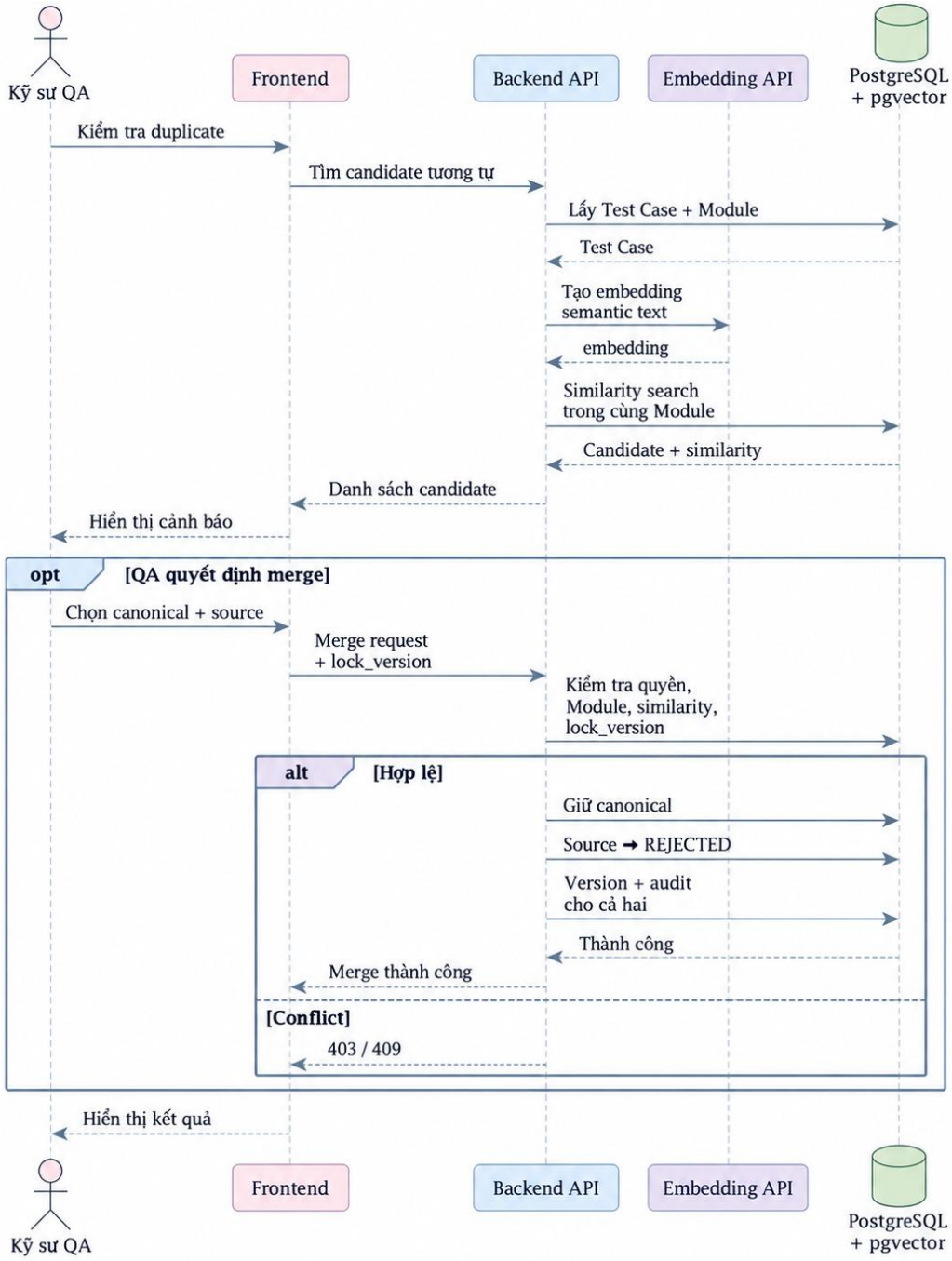


Hình 3.6: Sơ đồ tuần tự "Rà soát và duyệt Test Case"

3.5.2.3. Phát hiện trùng lặp và merge Test Case

Hình 3.7 mô tả quá trình phát hiện Test Case tương tự và thao tác merge. Hệ thống tạo embedding, tìm candidate trong cùng Module và trả similarity cho người

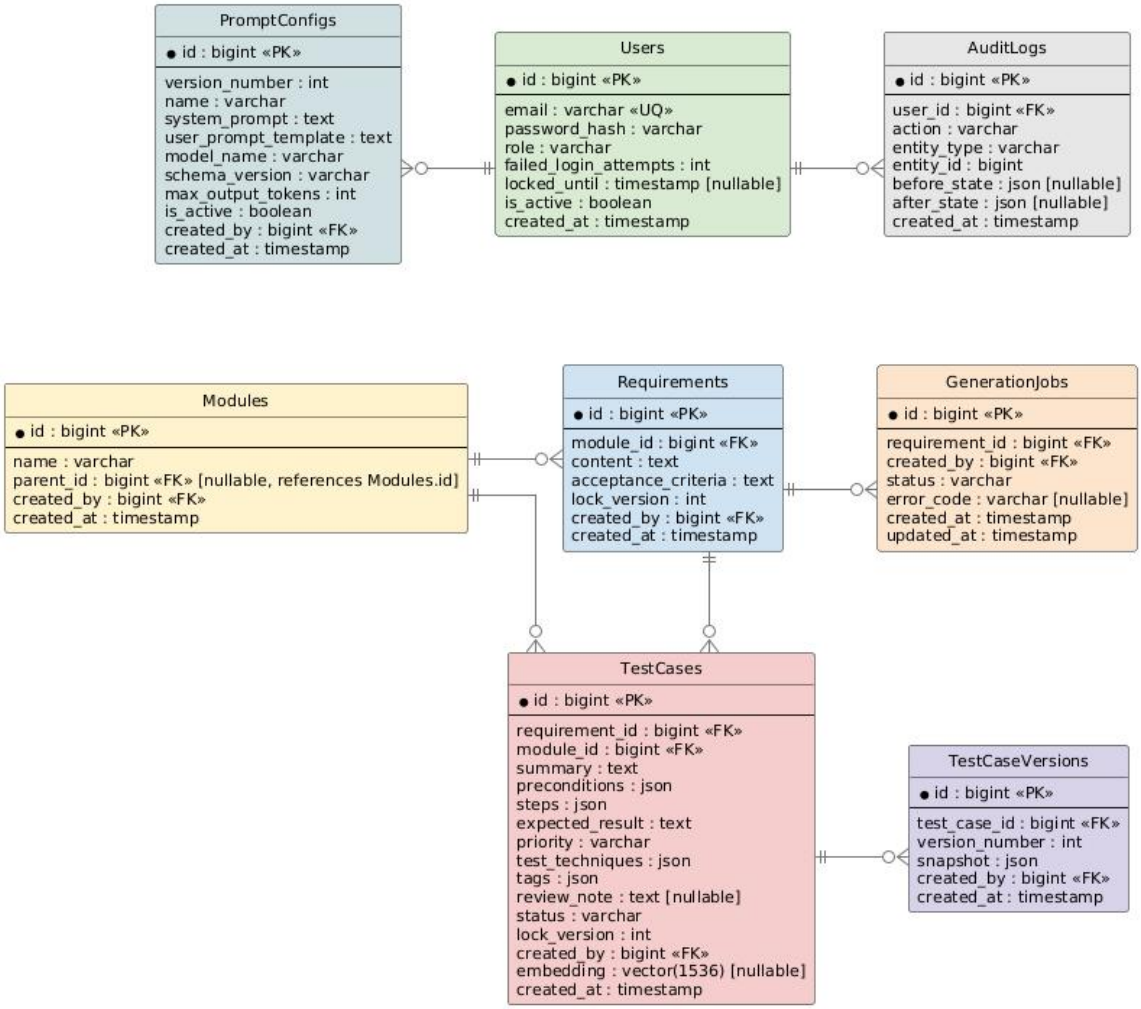
dùng. Nếu QA xác nhận merge, Test Case canonical được giữ lại còn Test Case nguồn chuyển sang REJECTED; version và audit của cả thao tác được lưu để truy vết.



Hình 3.7: Sơ đồ tuần tự "Phát hiện trùng lặp ngữ nghĩa và merge Test Case"

3.6. Thiết kế dữ liệu và lớp (ERD & Class Diagram)

3.6.1. Mô hình thực thể ERD



Hình 3.8: Mô hình thực thể ERD

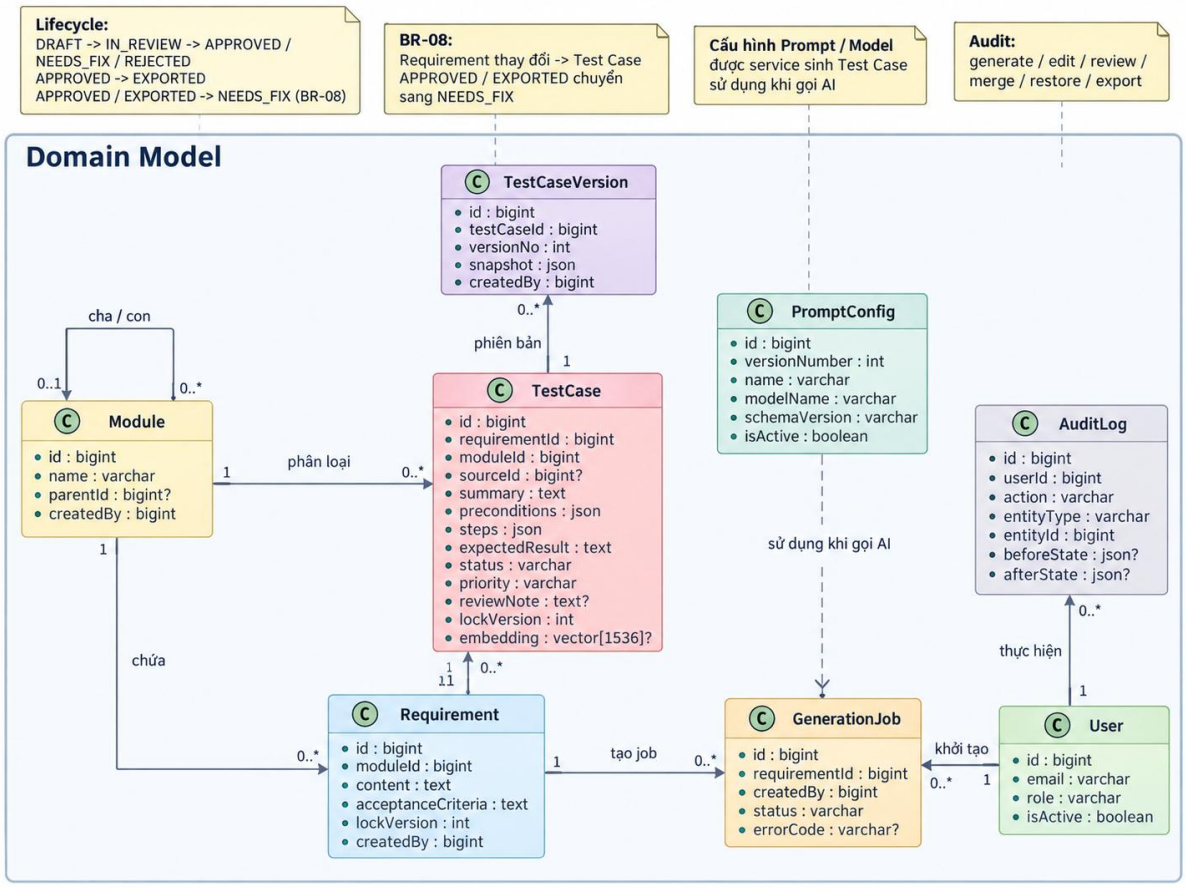
Hình 3.8 mô tả mô hình dữ liệu của hệ thống gồm tám thực thể chính: Users, Modules, Requirements, GenerationJobs, TestCases, TestCaseVersions, PromptConfigs và AuditLogs. Requirement thuộc một Module và là nguồn để tạo GenerationJob cũng như các Test Case; mỗi Test Case đồng thời lưu requirement_id và module_id để bảo đảm khả năng truy vết, và có thể có nhiều phiên bản trong TestCaseVersions. PromptConfigs quản lý phiên bản cấu hình Prompt/Model, trong khi AuditLogs lưu lịch sử thao tác của người dùng. Các khóa ngoại created_by đều tham chiếu Users.id nhưng các đường liên kết lặp lại này được lược khỏi hình để giảm nhiễu thị giác; quan hệ module cha–con được thể hiện trực tiếp qua Modules.parent_id.

Bảng	Trường cốt lõi đề xuất	Mục tiêu BR/NC
users	id, email, password_hash, role, failed_login_attempts, locked_until, is_active, created_at	NC-10, BR-05/07

Bảng	Trường cốt lõi đề xuất	Mục tiêu BR/NC
modules	id, name, parent_id, created_by, created_at	NC-06, BR-03
requirements	id, module_id, content, acceptance_criteria, lock_version, created_by, created_at	NC-01/08, BR-03/08
generation_jobs	id, requirement_id, created_by, status, error_code, created_at, updated_at.	NC-02/04/07, BR-01/02
test_cases	id, requirement_id, module_id, summary, preconditions, steps, expected_result, priority, test_techniques, tags, review_note, status, lock_version, created_by, created_at, embedding	BR-03 traceability
test_case_versions	id, test_case_id, version_number, snapshot, created_by, created_at	NC-08, BR-06
prompt_configs	id, version_number, name, system_prompt, user_prompt_template, model_name, schema_version, max_output_tokens, is_active, created_by, created_at	NC-09
audit_logs	id, user_id, action, entity_type, entity_id, before_state, after_state, created_at	NC-11, BR-06

Bảng 3.5: Cấu trúc dữ liệu đề xuất

3.6.2. Sơ đồ lớp



Hình 3.9: Sơ đồ lớp miền sinh và rà soát Test Case

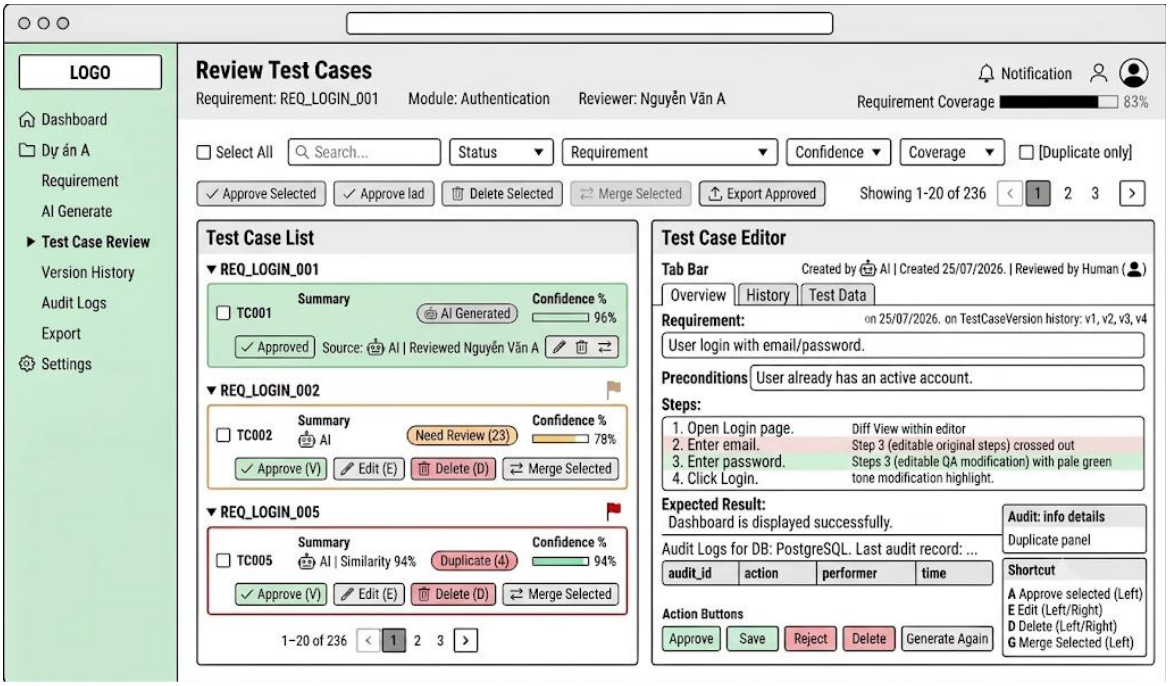
Hình 3.9 mô tả các lớp nghiệp vụ chính và quan hệ giữa các nhóm chức năng của hệ thống. Nhóm Requirement và Module quản lý dữ liệu đầu vào và khả năng truy vết; nhóm Test Case Lifecycle chịu trách nhiệm tạo Generation Job, sinh Test Case, review, quản lý phiên bản, duplicate và export; nhóm AI/Background Processing cô lập xử lý bất đồng bộ và kết nối với provider; Prompt Configuration, Audit và Identity & Access cung cấp các chức năng xuyên suốt.

Ở mức domain, một Module có nhiều Requirement và Test Case; một Requirement có thể tạo nhiều Generation Job và nhiều Test Case; mỗi Test Case có thể có nhiều TestCaseVersion. Các quan hệ này nhất quán với ERD và schema PostgreSQL hiện hành.

3.7. Thiết kế giao diện

Thiết kế giao diện tập trung vào màn hình Review Test Case vì đây là nơi thực hiện phần lớn thao tác human-in-the-loop. Wireframe ở Hình 3.10 tổ chức giao diện

thành hai vùng chính: danh sách Test Case và bộ lọc ở bên trái/trên, khu vực xem và chỉnh sửa chi tiết Test Case ở bên phải.



Hình 3.10: Wireframe định hướng màn hình rà soát Test Case

Người dùng có thể xem Requirement nguồn, nội dung hiện tại, trạng thái và thông tin duplicate; các thao tác Submit Review, Approve, Request Fix, Reject và Export được bố trí theo trạng thái và quyền người dùng. Wireframe này là thiết kế định hướng; giao diện chạy thật và bằng chứng nghiệp vụ được trình bày ở Chương 4.

3.8. Ma trận truy vết (Traceability Matrix)

Ma trận truy vết trong Bảng 3.6 được sử dụng để kiểm tra tính nhất quán xuyên suốt giữa BRD, thiết kế, cài đặt và kiểm thử. Mỗi dòng ánh xạ một nhóm NC/BR sang yêu cầu chức năng YC, Use Case thực hiện, Service/API tương ứng, dữ liệu hoặc trạng thái bị tác động và nhóm Test Case dùng để xác minh. Nhờ đó có thể truy ngược từ một quy tắc nghiệp vụ đến vị trí triển khai và bằng chứng kiểm thử.

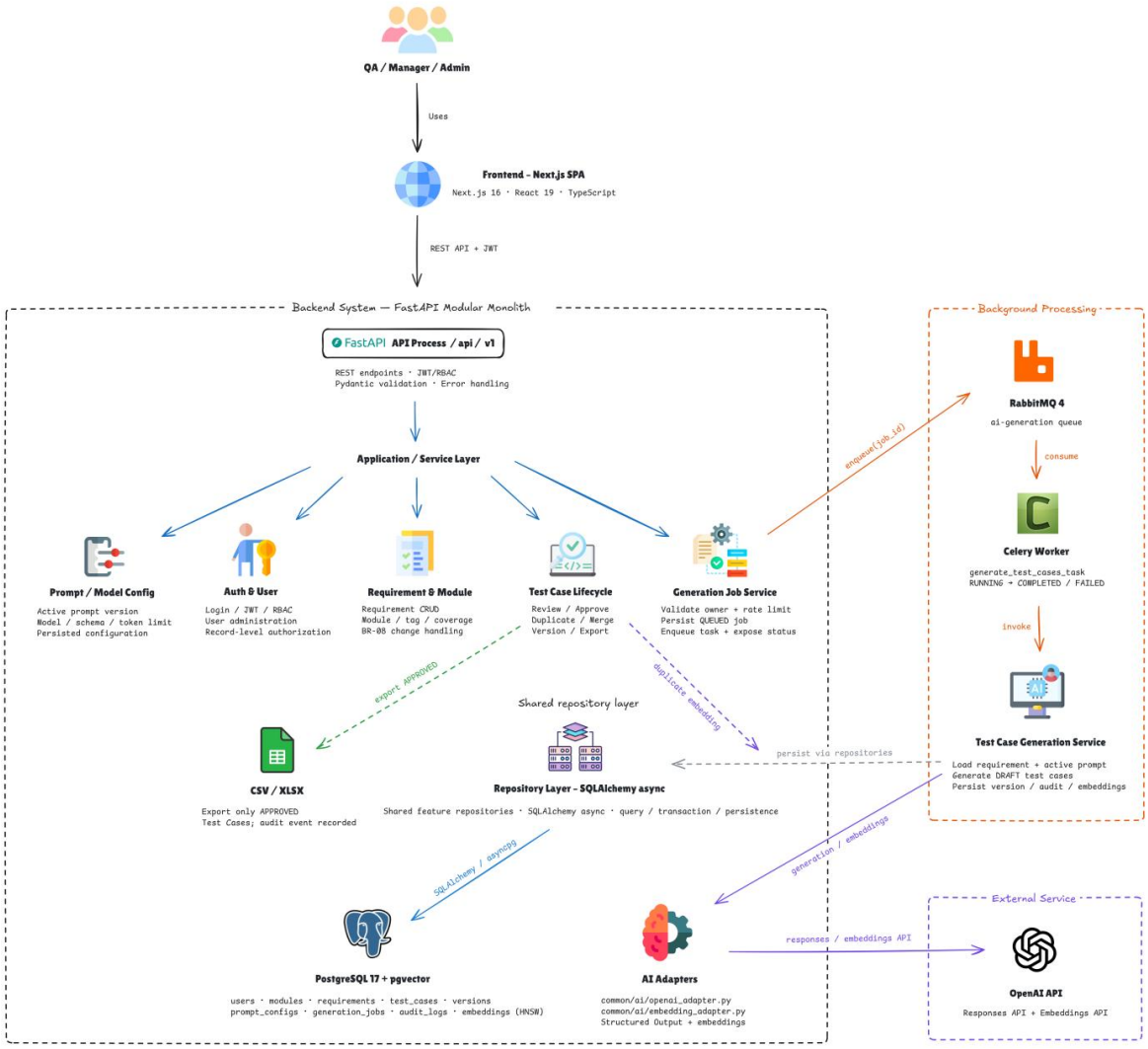
NC/BR	YC	UC	Service/API triển khai	Dữ liệu/State	Test ID dự kiến
NC-01, BR-07, BR-08	YC-01	UC05	RequirementService / /api/v1/requirements	Requirements; module_id; created_by; lock_version; Requirement thay đổi → Test Case APPROVED/EXPORTED → NEEDS_FIX	TC-REQ-012 → TC-REQ-017
NC-02, NC-03, BR-02, BR-03, BR-04, BR-06, BR-09	YC-02	UC06	GenerationJobService + TestCaseGenerationService + common/ai/OpenAIAdapter	generation_jobs: QUEUED → RUNNING → COMPLETED/FAILED; Test Case mới DRAFT; requirement_id, module_id; Structured Output; version + audit	TC-GEN-004 → TC-GEN-007
NC-04, NC-05, BR-01, BR-05, BR-06, BR-09	YC-03	UC07, UC08	Review/approval trong app/testcases	DRAFT → IN_REVIEW → APPROVED / NEEDS_FIX / REJECTED; lock_version; TestCaseVersion; audit	TC-REV-001 → TC-REV-007; TC-APP-001 → TC-APP-004
NC-05, BR-01, BR-06	YC-03	UC07	DuplicateDetectionService + DuplicateMergeService	embedding vector(1536); similarity threshold 0.85; candidate trong cùng Module; canonical giữ lại, source → REJECTED; version + audit	TC-DUP-001 → TC-DUP-005

NC/BR	YC	UC	Service/API triển khai	Dữ liệu/State	Test ID dự kiến
NC-06, NC-12, BR-03	YC-04	UC04, UC11	Module/Coverage trong app/modules	modules; Requirement/Test Case liên kết qua module_id và requirement_id; thống kê số Requirement/Test Case và Coverage theo Module	TC-MOD-001 → TC-MOD-005; TC-COV-001 → TC-COV-003
NC-07, BR-01, BR-05, BR-06	YC-04	UC09	ExportService	Chỉ Test Case APPROVED được export; sau export APPROVED → EXPORTED; CSV/XLSX; AuditLog export	TC-EXP-001 → TC-EXP-005
NC-08, BR-06, BR-08	YC-05	UC10, UC05	Version handling trong app/testcases + RequirementService cho BR-08	test_case_versions; snapshot; version_number; compare/restore; restore tạo version mới; Requirement đổi → liên quan NEEDS_FIX	TC-VER-001 → TC-VER-005; TC-REQ-017
NC-09	YC-05	UC03	Prompt configuration trong app/prompt_configs	prompt_configs; active version; system_prompt; user_prompt_template; model_name; schema_version	TC-PROMPT-001 → TC-PROMPT-003

NC/BR	YC	UC	Service/API triển khai	Dữ liệu/State	Test ID dự kiến
NC-10, BR-05, BR-07	YC-05	UC01, UC02	Auth/User trong app/auth, app/users	users; JWT/RBAC; role; record ownership; is_active; unauthorized → 401/403	TC-AUTH-011 → TC-AUTH-014; TC-USER-012 → TC-USER-017; TC-AUTHZ-001 → TC-AUTHZ-003
NC-11, BR-06	YC-05	Cross-cutting: UC03, UC05, UC10	Audit logging trong app/audit	audit_logs; action; entity_type; entity_id; before_state; after_state; audit generate/edit/approve/merge/export/BR-08	TC-AUD-001…

Bảng 3.6: Ma trận truy vết mở rộng

3.9. Thiết kế kiến trúc



Hình 3.11: Sơ đồ kiến trúc

Layer/Component	Thành phần	Trách nhiệm
Presentation	Next.js SPA	UI; loading/data/empty/error; gọi API qua client tập trung.
API	FastAPI routers /api/v1	HTTP/DTO/auth dependency; không SQL và không BR.
Application Service	Requirement/Generation/ Review/Duplicate/Export/ Auth/Audit	Thực thi BR, state transition, orchestration.
AI Adapter	common/ai/ OpenAIAdapter	Một điểm gọi OpenAI; Structured Outputs; timeout/error mapping; model config.

Layer/Component	Thành phần	Trách nhiệm
Repository	Feature repositories	SQLAlchemy query/transaction; không quyết định nghiệp vụ.
Database	PostgreSQL	User, module, Requirement, Test Case, version, prompt config, audit.
External	OpenAI API	LLM provider; không được xem là actor nội bộ của hệ thống.
Background Worker	Celery worker	Thực thi Generation Job nền, quản lý vòng đời QUEUED/RUNNING/COMPLETED/FAILED.
Message Broker	RabbitMQ	Truyền task generation giữa API và Celery worker; tách request web khỏi tác vụ AI có độ trễ.

Bảng 3.7: Thành phần kiến trúc đề xuất

CHƯƠNG 4: CÀI ĐẶT & TRIỂN KHAI

4.1. Môi trường & công nghệ

Bảng 4.1 ghi các phiên bản được xác nhận trên môi trường fresh clone độc lập ngày 16/09/2026 và từ package manifest của repository. Fresh clone chạy trên Windows bằng Git Bash với Python 3.13.15, Node.js 24.19.0 và npm 11.17.0; các dependency backend/frontend được cài lại từ đầu thành công. Kết quả CI run #91 ở Tuần 09 được giữ như bằng chứng lịch sử của pipeline, còn số liệu kiểm thử cuối cùng của bản bàn giao được trình bày ở Chương 5.

Thành phần	Phiên bản xác nhận	Vai trò trong đề tài
OS phát triển	Windows 10/11	Môi trường local chạy frontend, backend, PostgreSQL, RabbitMQ/Celery.
Python local	3.13.15	Backend FastAPI, test và worker.

Thành phần	Phiên bản xác nhận	Vai trò trong đề tài
FastAPI / Uvicorn	0.141.1 / 0.53.0	REST API, OpenAPI và server ASGI.
SQLAlchemy / asyncpg	2.0.53 / 0.31.0	ORM async và kết nối PostgreSQL.
Alembic	1.20.0	Migration schema từ 0001 đến 0010.
Pydantic	2.13.5	DTO, server validation và Structured Output schema.
OpenAI SDK	2.54.0	Responses API và embedding adapter.
PostgreSQL + pgvector	17.x + pgvector	Lưu dữ liệu quan hệ và vector embedding.
RabbitMQ / Celery	4.2.9 / 5.6.3	Broker và background generation job.
Node.js / npm	24.19.0 / 11.17.0	Runtime và dependency frontend.
Next.js / React	16.2.11 / 19.2.0	Web frontend.
TypeScript	5.9.x	Kiểu dữ liệu và logic frontend.
Pytest / Ruff	9.1.1 / 0.16.7	Automated test, lint và format backend.
Newman	6.2.2	API smoke test từ Postman collection; bằng chứng Tuần 09.

Bảng 4.1: Môi trường và phiên bản công nghệ của hệ thống

Giấy phép các thành phần chính đã được kiểm tra theo metadata/LICENSE của từng dự án: FastAPI (MIT), Uvicorn (BSD-3-Clause), SQLAlchemy (MIT), asyncpg (Apache-2.0), Alembic (MIT), Pydantic (MIT), OpenAI Python SDK (Apache-2.0), Celery (BSD-3-Clause), openpyxl (MIT), Next.js (MIT), React (MIT), TypeScript

(Apache-2.0), RabbitMQ (MPL-2.0), PostgreSQL và pgvector (PostgreSQL License). Nhóm thư viện lõi sử dụng trong đồ án không yêu cầu giấy phép GPL/AGPL.

Tên gọi giữa thiết kế và source được thống nhất như sau: Requirement ↔ app/requirements; Test Case ↔ app/testcases; Prompt/Model Configuration ↔ app/prompt_configs; User/RBAC ↔ app/auth và app/users; Module/Coverage ↔ app/modules; Audit Log ↔ app/audit; LLM/Embedding Adapter ↔ app/common/ai. Cách đối chiếu này giúp truy vết trực tiếp sơ đồ Chương 3 sang code.

4.2. Cấu trúc mã nguồn

Source code tách frontend và backend trong src/. Backend tổ chức theo feature, nhưng trong từng feature vẫn phân lớp Router → Service → Repository. Phần gọi mô hình được cô lập ở common/ai. Bản bàn giao bổ sung nhóm script ở thư mục scripts/ để tạo môi trường, migration/seed và khởi động hệ thống theo đường dẫn tương đối, nhờ đó không phụ thuộc ổ đĩa hay IDE cụ thể.

Cây thư mục hai cấp dùng để tra nhanh source:

```
src/
  frontend/                Next.js + TypeScript
  backend/
    app/                   auth, users, requirements, modules, testcases, audit, common/ai
    alembic/               migration database
    scripts/seed_demo.py   dữ liệu demo
scripts/
  install.sh               tạo .env/.venv và cài dependency
  init-demo.sh             migration + seed
  dev-up.sh / dev-down.sh  khởi động / dừng môi trường local
  python.sh                wrapper Python đa nền tảng
docs/kiem-thu/             Test Case, RTM, Gold Set, Newman
docs/assets/final/         ảnh minh họa bản bàn giao
.github/workflows/ci.yml   CI backend + frontend
```

Thư mục/tệp	Trách nhiệm	Tầng kiến trúc Chương 3
src/frontend	Giao diện QA/Manager/Admin, gọi REST API, trạng thái loading/error.	Presentation
app/*/router.py	Nhận HTTP, dependency/auth, chuyển DTO sang Service.	Router/API
app/*/service.py	Business rule, authorization record-level, state transition, orchestration.	Service
app/*/repository.py	SQLAlchemy query và persistence.	Repository
app/common/ai	OpenAI Responses API và embedding.	External adapter
app/worker.py + task	Celery worker xử lý generation bất đồng bộ.	Background worker
alembic + database/schema.sql	Migration và snapshot schema.	Data
scripts/ + docs/kiem-thu	Reproducible local setup; Test Case, RTM, regression, Gold Set và Newman.	Deployment / Verification

Bảng 4.2: Trách nhiệm các khu vực mã nguồn và ánh xạ kiến trúc

Ví dụ luồng Requirement: router nhận payload đã được Pydantic validate, RequirementService kiểm role / ownership / lock_version và BR - 08, RequirementRepository / TestCaseRepository thao tác dữ liệu; router không truy cập database trực tiếp. Đây là cấu trúc triển khai tương ứng kiến trúc đã trình bày ở Chương 3.

4.3. Cài đặt database

ERD Chương 3 được hiện thực bằng PostgreSQL và Alembic. Schema chạy thật hiện có tám bảng nghiệp vụ chính. Không tồn tại bảng test_case_requirements trung gian: mỗi Test Case giữ requirement_id trực tiếp vì phạm vi hiện tại yêu cầu truy vết tối thiểu một yêu cầu nguồn và hệ thống đang dùng quan hệ một Requirement – nhiều Test Case.

Thực thể ERD	Bảng thật	PK/FK chính	Ghi chú triển khai
User	users	id; email UNIQUE	role; password_hash; failed_login_attempts; locked_until; is_active.
Module	modules	id; parent_id → modules; created_by → users	Cây module; unique tên không phân biệt hoa thường theo parent.
Requirement	requirements	id; module_id → modules; created_by → users	content, acceptance_criteria, lock_version.
TestCase	test_cases	id; requirement_id → Requirements; module_id → modules; created_by → users	status, priority, JSON steps/tags/techniques, lock_version, embedding vector(1536).
GenerationJob	generation_jobs	id; requirement_id; created_by	QUEUED/RUNNING/COMPLETED/FAILED, error_code.
TestCaseVersion	test_case_versions	id; test_case_id → test_cases	version_number + snapshot JSON; UNIQUE(test_case_id, version_number).
PromptConfig	prompt_configs	id; created_by →	version,

Thực thể ERD	Bảng thật	PK/FK chính	Ghi chú triển khai
		users	prompt/template, model, schema, max tokens, active flag.
AuditLog	audit_logs	id; user_id → users	action, entity_type/id, before_state, after_state, created_at.

Bảng 4.3: Ánh xạ ERD sang schema PostgreSQL hiện hành

Migration được quản lý bằng Alembic thay vì sửa bảng thủ công. Chuỗi hiện tại đi từ 0001_week05_baseline tới 0010_module_name_unique. Các mốc chính gồm generation_jobs (0002), HITL/version (0003), pgvector + HNSW (0004), export (0005), tag/Coverage (0006), prompt config (0007), Requirement lock_version + restore (0008), user administration (0009) và unique module name (0010).

Từ thư mục gốc, migration và seed được gom vào một lệnh bàn giao:

```
./scripts/init-demo.sh
```

Seed final tạo 3 tài khoản demo, 2 Module, 3 Requirement và 12 Test Case có trạng thái/priority đa dạng. Kiểm thử integration dùng database riêng testcase_ai_test; database demo/dev là testcase_ai. Guard trong test chặn trường hợp integration chạy nhầm vào database demo.

4.4. Cài đặt các chức năng chính

Các mục dưới đây mở đầu bằng mã Use Case, yêu cầu (YC – tương đương FR trong SRS của đề tài) và Business Rule. Mỗi mục trình bày đầu vào/đầu ra, luồng xử lý, đoạn mã then chốt hoặc điểm kiểm soát, và bằng chứng chạy thật. Các CRUD tương tự được gộp để tránh lặp.

Mục	UC/YC/BR chính	Chức năng hiện thực
4.4.1	UC01–UC03; YC-05; BR-05, BR-07	Đăng nhập, RBAC, quản lý user, Prompt/Model Configuration.
4.4.2	UC04–UC05; YC-01/YC-04; BR-03, BR-07, BR-08	Module, Requirement, reopen/update Requirement và revalidation.
4.4.3	UC07–UC08-UC10; YC-03; BR-01, BR-05, BR-06, BR-09	HITL review/edit/approve/reject, version, duplicate.
4.4.4	UC04-UC09-UC11 + NC-06/NC-07/NC-12; BR-03, BR-05, BR-06	Coverage, module, tag và export CSV/XLSX.
4.5	UC06; YC-02; BR-02, BR-04, BR-06	AI generation bất đồng bộ, Structured Output, retry, pgvector.

Bảng 4.4: Truy vết chức năng triển khai sang UC/YC/BR

4.4.1. Xác thực, quản lý người dùng và cấu hình Prompt/Model

Mục này hiện thực UC01–UC03, tương ứng YC-05/NC-09/NC-10 và kiểm tra BR-05, BR-07. Admin quản lý tài khoản và cấu hình Prompt/Model; mọi endpoint vẫn kiểm role ở backend, không dựa vào việc ẩn nút trên UI.

Nội dung	Đầu vào	Kết quả hợp lệ	Lỗi/điểm chặn
Đăng nhập	email + password	JWT có sub, role, exp	401 khi sai thông tin; 403 nếu tài khoản inactive; 429 khi tài khoản đang bị khóa.
Quản lý user	email, role, trạng thái active	Tạo/cập nhật user; audit	403 nếu không phải Admin; response không trả password_hash.
Prompt config	system prompt,	Tạo version mới và	422 nếu thiếu

Nội dung	Đầu vào	Kết quả hợp lệ	Lỗi/điểm chặn
	template, model, schema, max tokens	active	{Requirement_text}/{acceptance_criteria} hoặc placeholder không hỗ trợ; 403 sai role.

Bảng 4.5: Đầu vào / đầu ra cho nhóm UC01 - UC03

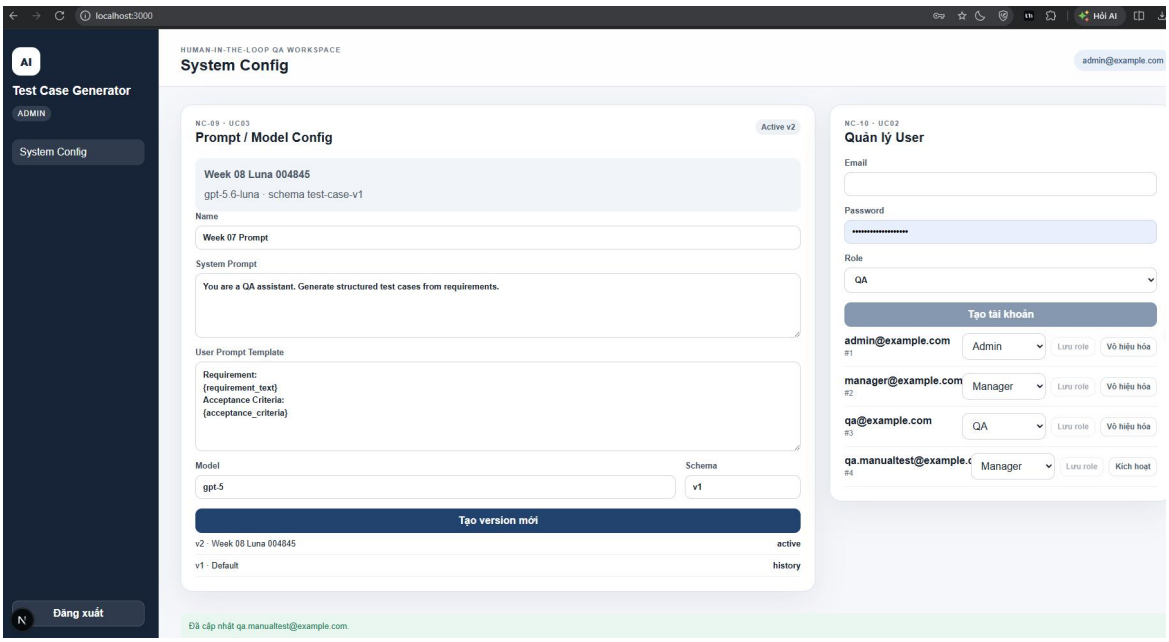
Luồng xử lý: frontend gửi request → FastAPI/Pydantic kiểm DTO → Service kiểm role → repository đọc/ghi user hoặc prompt_configs → audit ghi before/after phù hợp → trả DTO đã loại dữ liệu nhạy cảm. Password được băm bcrypt cost 12; JWT chỉ chứa claim định danh/role/exp.

Đoạn mã then chốt – validate Prompt Template trước khi lưu:

```
required = {"{Requirement_text}", "{acceptance_criteria}"}
missing = [p for p in required if p not in self.user_prompt_template]
if missing:
    raise ValueError("template must include required placeholders")
self.user_prompt_template.format(
    Requirement_text="REQ", acceptance_criteria="AC"
)
```

Mã 4.1: Kiểm tra placeholder bắt buộc của Prompt Template

Hình 4.1 cho thấy màn System Config đang dùng thật: active Prompt/Model v2, model gpt-5.6-luna, schema test-case-v1 và khu vực quản lý user theo role. Cần chú ý cấu hình prompt được version hóa thay vì hardcode trong luồng nghiệp vụ.



Hình 4.1: System Config - Prompt / Model version và quản lý user theo role

4.4.2. Module, Requirement và cập nhật theo BR-08

Mục này hiện thực UC04–UC05, tương ứng YC-01/YC-04, NC-01/NC-06 và BR-03, BR-07, BR-08. QA chỉ tạo/cập nhật Requirement của mình; Manager quản lý Module; Requirement lưu module_id và lock_version.

Đầu vào	Xử lý chính	Đầu ra	Lỗi/điểm chặn
module_id, content/SRS, acceptance_criteria	Validate module + role/owner	Requirement v1 hoặc version mới	404 module/Requirement không tồn tại; 403 sai owner.
requirement_id + lock_version khi cập nhật	Optimistic locking; đánh dấu TC liên quan NEEDS_FIX	Requirement tăng lock_version; audit + version TC	409 nếu client dùng version cũ.

Bảng 4.6: Đầu vào/đầu ra của Requirement và BR-08

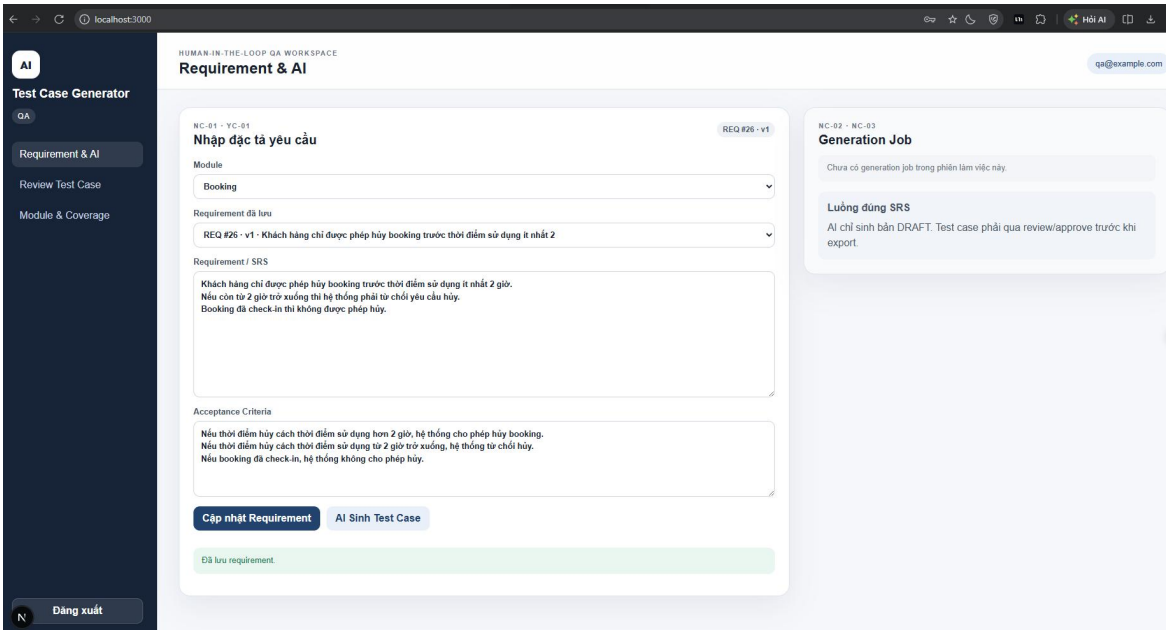
Luồng cập nhật đi qua RequirementService: service khóa record để cập nhật, kiểm created_by và lock_version; sau một PATCH hợp lệ, các Test Case APPROVED/EXPORTED liên quan được chuyển NEEDS_FIX, tăng lock_version, tạo snapshot version và ghi audit. Đây là hiện thực trực tiếp chính sách BR-08 theo hướng an toàn, buộc rà soát lại sau khi Requirement nguồn được chỉnh sửa.

Đoạn mã then chốt – ownership, optimistic locking và BR-08:

```
if Requirement.created_by != current_user.id:
    raise AppError(ErrorCode.FORBIDDEN_RECORD, ..., 403)
if Requirement.lock_version != payload.lock_version:
    raise AppError(ErrorCode.CONFLICT, ..., 409)
affected_ids = await self._mark_related_cases_for_review(Requirement.id,
current_user.id)
Requirement.lock_version += 1
...
# BR-08: approved/exported cases become NEEDS_FIX
for test_case in records:
    test_case.status = TestCaseStatus.NEEDS_FIX
    test_case.lock_version += 1
```

Mã 4.2: Kiểm tra ownership, optimistic locking và áp dụng BR-08

Hình 4.2 chứng minh QA có thể chọn lại Requirement đã lưu theo Module – chức năng được bổ sung sau defect #26. Đây là điểm cần nhìn vì trước fix giao diện chỉ giữ form tạo mới.



Hình 4.2: QA mở lại Requirement đã lưu theo Module để cập nhật

4.4.3. Rà soát, duyệt, duplicate và lịch sử phiên bản

Mục này hiện thực UC07–UC08, tương ứng YC-03/NC-04/NC-05/NC-08 và BR-01, BR-05, BR-06, BR-09. Test Case AI sinh luôn bắt đầu ở DRAFT; người dùng phải đưa qua quy trình HITL trước khi sử dụng/export.

Nhóm	Luồng/đầu vào	Kết quả chính	Kiểm soát
HITL	DRAFT → IN_REVIEW → APPROVED/NEEDS_FIX/REJECTED	status + lock_version + version snapshot	Sai transition/role/record bị chặn.
Version	Chọn version cũ, compare/restore	Snapshot mới sau restore	Optimistic locking + audit.
Duplicate	Test Case mục tiêu trong cùng module	Danh sách candidate + similarity	QA chỉ thấy record của mình; threshold 0.85.
Merge duplicate	Chọn canonical/source candidate	Canonical giữ lại; source → REJECTED	Ghi version + audit, không tạo quan hệ giả.

Bảng 4.7: Đầu vào/đầu ra cho HITL, version và duplicate

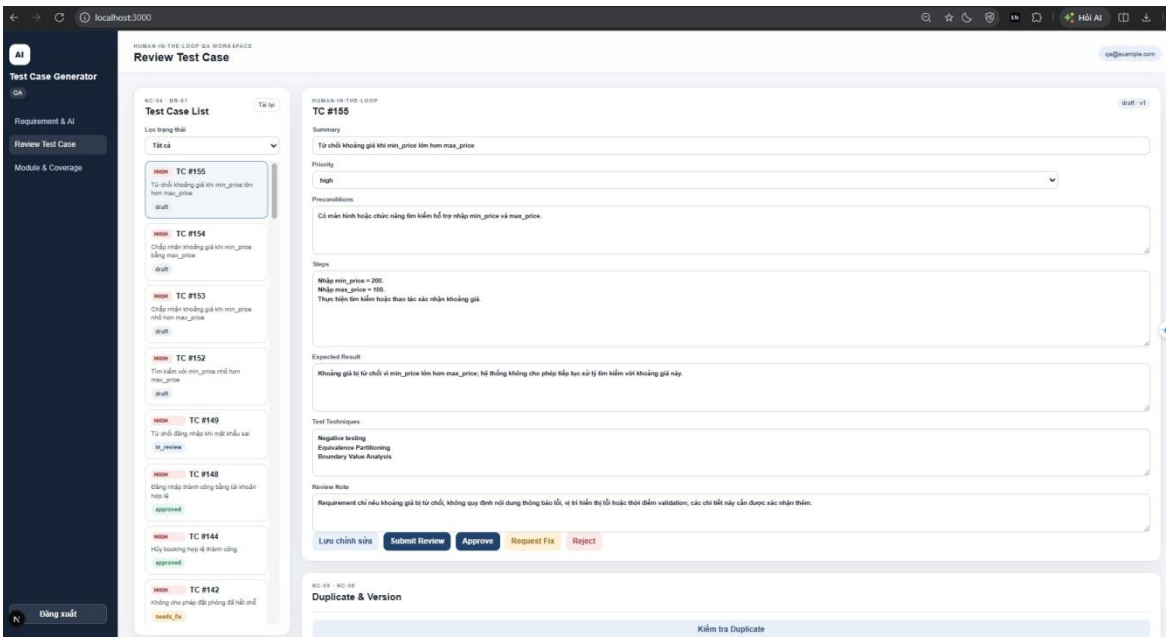
Luồng xử lý: frontend gửi thao tác review/approve/request-fix/reject hoặc merge duplicate → FastAPI nhận DTO → Service kiểm role, quyền trên record và lock_version → Repository cập nhật trạng thái/merge → hệ thống tạo TestCaseVersion và AuditLog → commit transaction → trả DTO mới về giao diện.

Đoạn mã then chốt – chặn duyệt sai trạng thái và ghi nhận transition:

```
if test_case.status != TestCaseStatus.IN_REVIEW:
    raise self._conflict("Chỉ Test Case IN_REVIEW mới có thể được duyệt.")
self._validate_required_fields(test_case)
return await self._transition(
    test_case, current_user, TestCaseStatus.APPROVED,
AuditAction.APPROVE_TEST_CASE
)
```

Mã 4.3: Kiểm tra trạng thái trước khi duyệt Test Case

Hình 4.3 là màn Review Test Case thực tế của bản final. Điểm cần nhìn là editor đã có đủ Summary, Priority, Preconditions, Steps, Expected Result, Test Techniques và Review Note; các nút Submit Review, Approve, Request Fix và Reject thể hiện vòng đời HITL, còn khu Duplicate & Version hỗ trợ kiểm tra candidate và lịch sử trước khi quyết định.



Hình 4.3: Màn Review Test Case với HITL và Duplicate & Version

Hình 4.4 cho thấy version history/restore đã chạy với snapshot dữ liệu thật. Chức năng này phục vụ NC-08 và BR-06, đồng thời giúp truy vết thay đổi sau edit/approve/restore.



Hình 4.4: Lịch sử phiên bản và khôi phục Test Case

4.4.4. Coverage, tag và export CSV/XLSX

Mục này hiện thực phần quản lý của UC04 và UC09, tương ứng NC-06/NC-07/NC-12 và BR-03, BR-05, BR-06. Coverage được tính theo Requirement/Test Case của Module; export chỉ lấy Test Case APPROVED và kiểm quyền ở backend.

Đầu vào	Xử lý	Đầu ra	Lỗi/điểm chặn
module_id	Tổng hợp Requirement/Test Case/APPROVED	Coverage theo module	404 module không tồn tại; role bị kiểm ở server.
format=csv/xlsx + module	Lọc Test Case APPROVED; tạo file; ghi audit	CSV hoặc XLSX đúng cột	403 user không có quyền export; không xuất DRAFT.

Bảng 4.8: Đầu vào/đầu ra của Coverage và Export

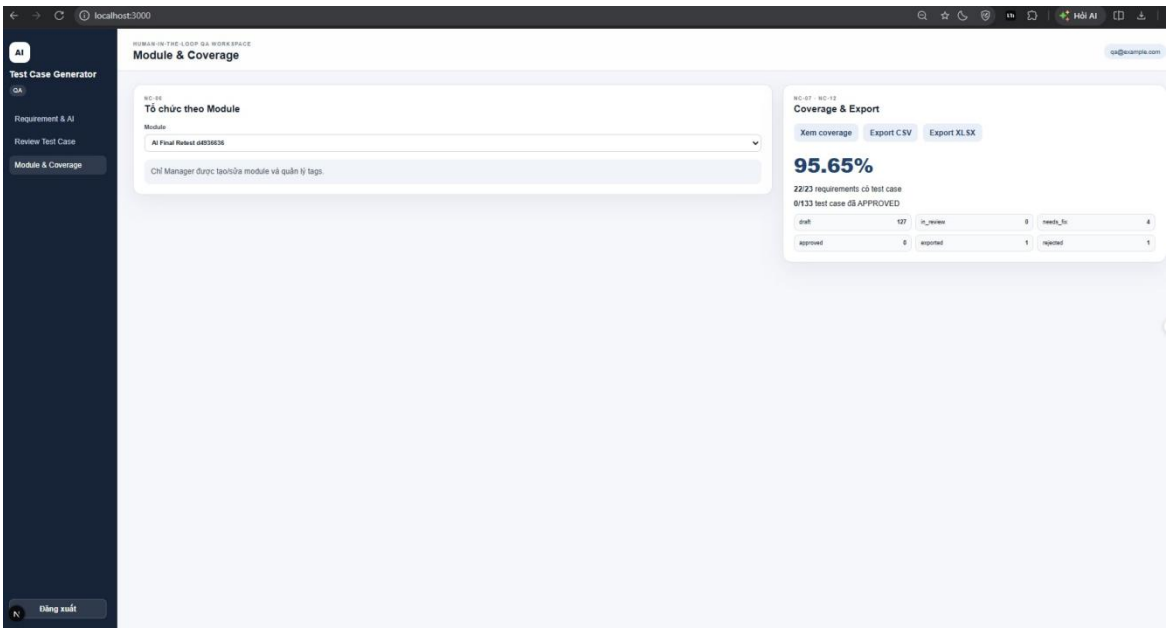
Luồng xử lý export: frontend gửi module_id và định dạng → FastAPI xác thực người dùng → ExportService kiểm role QA/Manager, module tồn tại và quyền ownership → Repository chỉ lấy Test Case APPROVED → bộ tạo file sinh CSV/XLSX → Test Case được chuyển EXPORTED → AuditLog ghi module, định dạng, số lượng và danh sách id → commit.

Đoạn mã then chốt – chỉ export dữ liệu đã duyệt và có quyền:

```
self._require_export_role(current_user)
test_cases = await self._load_approved_test_cases(module_id, current_user)
self._validate_approved_records(test_cases)
exported_file = self._build_file(module_id, export_format, test_cases)
self._mark_exported(test_cases)
```

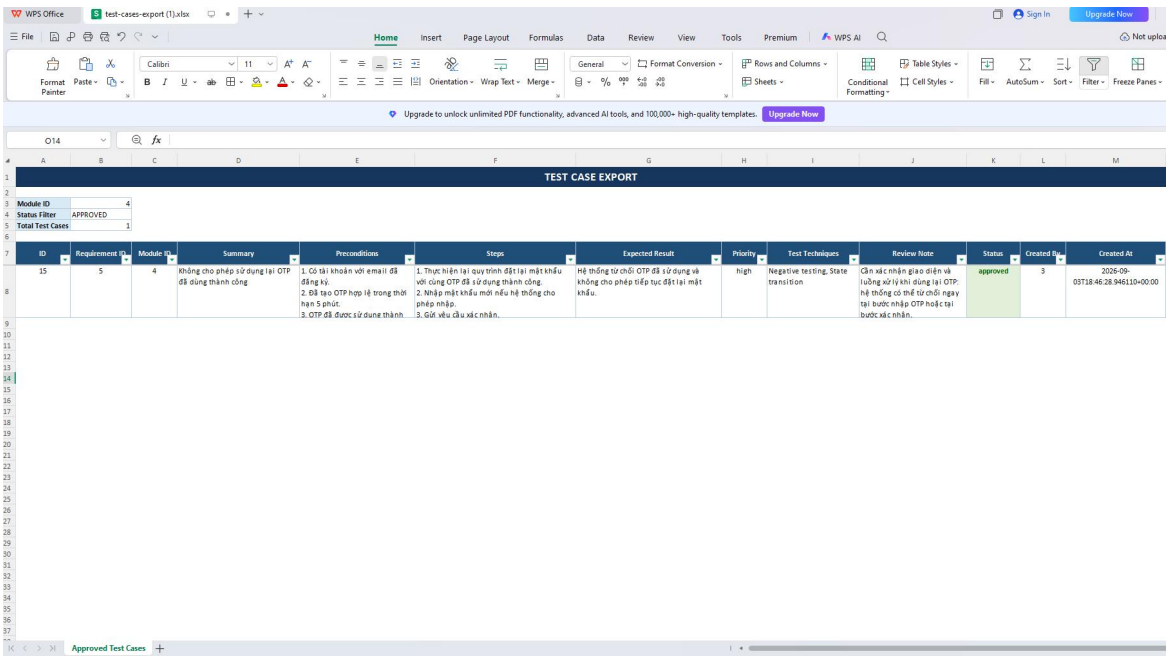
Mã 4.4: Kiểm tra quyền và lọc Test Case APPROVED trước khi export

Hình 4.5 là snapshot Coverage trên dữ liệu demo/final. Tại thời điểm chụp, Requirement Coverage đạt 95,65% trong Module đang chọn; giao diện đồng thời hiển thị phân bố Test Case theo trạng thái và cung cấp điểm vào Export CSV/XLSX. Con số này là coverage của dữ liệu demo tại thời điểm chụp, không được dùng thay cho Full Coverage Rate của Gold Set ở Chương 5.



Hình 4.5: Coverage theo Module và điểm vào Export

Hình 4.6 chứng minh file XLSX export mở được bằng ứng dụng bảng tính và chứa các cột nghiệp vụ: Requirement/module, summary, preconditions, steps, expected result, priority, techniques, review note và status. Dòng minh họa có status APPROVED, phù hợp rule chỉ export Test Case đã duyệt.



Hình 4.6: File XLSX export từ bộ Test Case APPROVED

4.4.5. Sinh Test Case bằng AI và theo dõi Generation Job

Mục này hiện thực UC06, tương ứng YC-02/NC-02/NC-03 và BR-02, BR-04, BR-06. QA gửi yêu cầu sinh cho Requirement thuộc quyền sở hữu; API không chờ mô hình trả xong mà tạo Generation Job và trả HTTP 202 để frontend theo dõi trạng thái.

Phần	Nội dung triển khai
Gắn mã	UC06; YC-02; BR-02 (đủ trường), BR-04 (schema/retry), BR-06 (version/audit).
Đầu vào / đầu ra	requirement_id của QA → HTTP 202 + job_id/status; polling /generation-jobs/{id} → QUEUED/RUNNING/COMPLETED/FAILED.
Luồng xử lý	Frontend → FastAPI → GenerationJobService kiểm role/owner + rate limit → RabbitMQ → Celery → OpenAIAdapter → Structured Output validation → PostgreSQL lưu Test Case DRAFT/version/audit.
Trường hợp lỗi	403 sai owner/role; 404 Requirement không tồn tại; 429 vượt rate limit; 502 provider/schema lỗi sau retry.
Bằng chứng chạy	Hình 4.7 cho thấy Generation Job COMPLETED trên giao diện; Hình 4.8 cho thấy Test Case có thể được tạo thủ công ở DRAFT mà không gọi OpenAI.

Bảng 4.9: Khuôn năm phần cho UC06 – Sinh Test Case bằng AI

Ngoài luồng AI, bản final bổ sung đường tạo Test Case thủ công tại POST /requirements/{requirement_id}/test-cases/manual. Luồng này không gọi OpenAI, không tạo Generation Job, tạo Test Case ở trạng thái DRAFT và tạo Version 1 để tiếp tục cùng quy trình HITL. Đây là fallback phục vụ demo/bàn giao khi chưa cấu hình API key; lõi AI của UC06 vẫn được giải thích chi tiết ở mục 4.5.

4.5. Phần lõi kỹ thuật đặc thù

Phần lõi của đề tài không chỉ là "gọi OpenAI API", mà là làm cho đầu ra AI có thể kiểm soát, chạy bất đồng bộ, truy vết được và phát hiện trùng lặp. Ba rủi ro kỹ thuật chính là: (1) LLM có thể trả dữ liệu sai schema; (2) generation có độ trễ nên không nên giữ HTTP request chờ lâu; (3) Test Case sinh ra có thể gần trùng nhau. Giải pháp được chọn lần lượt là Structured Output + Pydantic retry, Generation Job qua RabbitMQ/Celery, và semantic duplicate bằng pgvector.

4.5.1. Prompt version + Structured Output và retry BR-04

Prompt được lưu trong prompt_configs gồm system_prompt, user_prompt_template, model_name, schema_version và max_output_tokens. Runtime lấy active config nên có thể đổi model/prompt mà không sửa logic Service. OpenAIAdapter dùng Responses API với text_format=GeneratedTestCaseBatch; chỉ output_parsed hợp lệ mới được đưa vào bước lưu Test Case.

Về lựa chọn kỹ thuật, phương án output tự do/JSON mode đơn thuần dễ tạo dữ liệu thiếu trường hoặc sai enum, khiến tầng Service phải tự suy đoán và làm tăng rủi ro lưu dữ liệu không hợp lệ. Vì vậy hệ thống chọn Structured Outputs kết hợp Pydantic: provider bị ràng buộc bởi schema, còn backend vẫn giữ một lớp validation độc lập trước khi persist.

Đoạn mã then chốt – retry đầu ra sai schema theo BR-04:

```
MAX_SCHEMA_RETRIES = 2
for attempt in range(MAX_SCHEMA_RETRIES + 1):
    try:
        response = await self._parse_response(
            user_prompt, prompt_config,
            schema_reminder=attempt > 0,
        )
    except ValidationError as exc:
        last_validation_error = exc
        continue
    if response.output_parsed is not None:
        return response
raise AppError(ErrorCode.AI_OUTPUT_INVALID, ..., 502)
```

Mã 4.5: Retry có giới hạn khi Structured Output không hợp lệ

MAX_SCHEMA_RETRIES=2 nghĩa là một lần gọi ban đầu và tối đa hai lần sinh lại khi validation lỗi. Lỗi provider khác schema được tách thành AI_PROVIDER_ERROR 502; nếu hết số lần retry mà vẫn sai schema, hệ thống trả AI_OUTPUT_INVALID 502 thay vì lưu dữ liệu không hợp lệ.

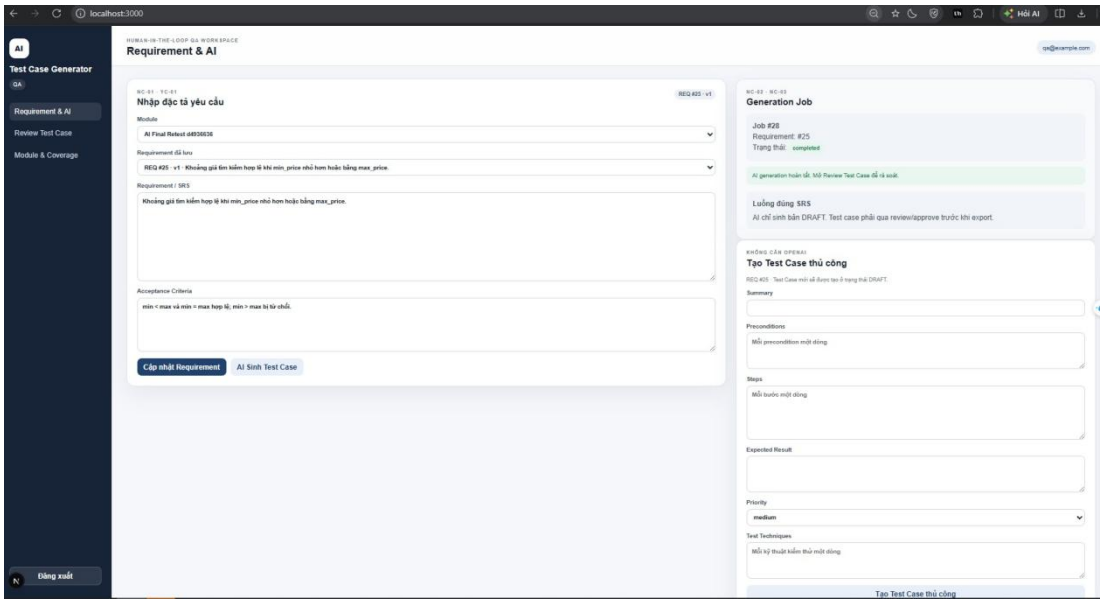
4.5.2. Generation Job bất đồng bộ với RabbitMQ/Celery

Cách đồng bộ (FastAPI chờ model trả xong rồi mới trả HTTP) được loại bỏ vì dễ timeout và làm request giữ tài nguyên lâu. Hệ thống tạo generation_jobs với QUEUED/RUNNING/COMPLETED/FAILED, trả 202 cho client, gửi task qua

RabbitMQ và để Celery worker gọi OpenAI rồi persist Test Case DRAFT. Frontend polling trạng thái job cho tới trạng thái cuối.

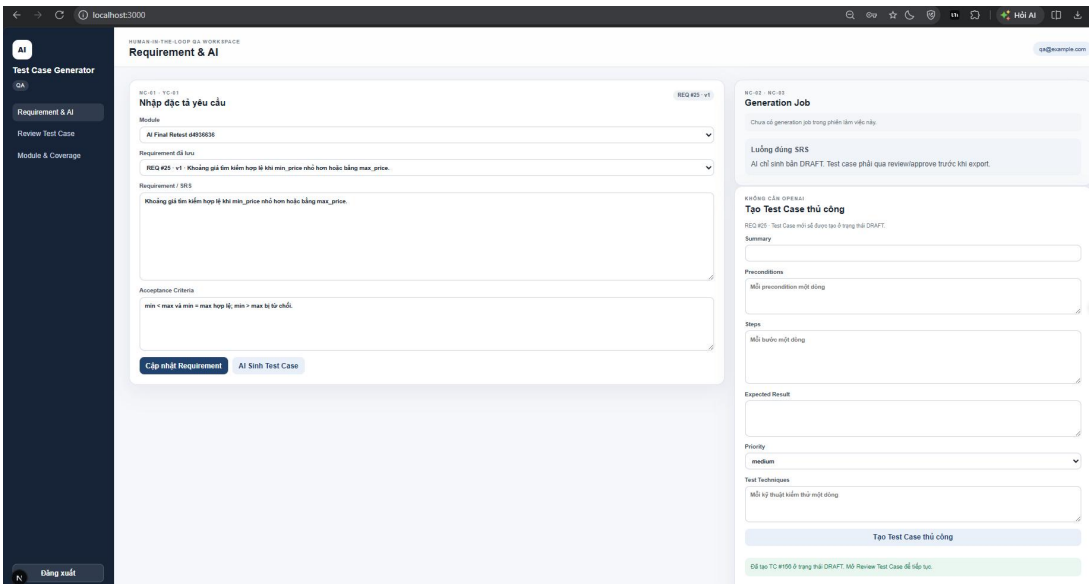
Defect #24 cho thấy trên Windows, việc tạo event loop mới cho từng task trong khi tái sử dụng async database resource có thể gây lỗi. Fix dùng persistent asyncio.Runner trong worker và được bổ sung automated regression, sau đó E2E RabbitMQ/Celery/OpenAI chạy lại PASS.

Hình 4.7 cho thấy luồng thật đi hết tới COMPLETED trên Requirement #25; giao diện hiển thị Generation Job hoàn tất và cho phép tiếp tục review Test Case được sinh.



Hình 4.7: Generation Job hoàn tất và Test Case được sinh chờ review

Hình 4.8 minh họa fallback không cần OpenAI trong cùng Requirement workspace. Sau khi người dùng nhập Summary, Preconditions, Steps, Expected Result, Priority và Test Techniques, hệ thống tạo TC #156 ở trạng thái DRAFT. Automated integration test xác nhận luồng này tạo Version 1 và không phát sinh generation_jobs.



Hình 4.8: Tạo Test Case thủ công và lưu ở trạng thái DRAFT

4.5.3. Semantic duplicate bằng pgvector

Duplicate Detection tạo semantic text từ Test Case, gọi embedding adapter, lưu vector 1536 chiều vào test_cases.embedding và truy vấn candidate trong cùng Module bằng cosine similarity. Mặc định dùng text-embedding-3-small, threshold 0.85 và HNSW index ix_test_cases_embedding_hnsw. Embedding bị vô hiệu hóa khi nội dung liên quan thay đổi và có script rebuild theo batch.

So khớp exact text hoặc hash chỉ phát hiện các bản sao gần như giống chữ và bỏ sót các Test Case diễn đạt khác nhau nhưng cùng ý nghĩa. Vì vậy hệ thống chọn embedding + cosine similarity trên pgvector, giới hạn candidate trong cùng Module và chỉ dùng ngưỡng 0,85 để cảnh báo; quyết định merge vẫn thuộc người dùng.

Đoạn mã then chốt – threshold và record-level visibility khi tìm duplicate:

```
settings = get_settings()
self._threshold = settings.duplicate_similarity_threshold
self._embedding_model = settings.openai_embedding_model
...
candidates, total = await self._test_cases.find_duplicate_candidates(
    test_case_id=target.id,
    module_id=target.module_id,
    owner_id=current_user.id if current_user.role == UserRole.QA else None,
    threshold=self._threshold,
    offset=offset,
    limit=limit,
)
```

Mã 4.6: Cấu hình ngưỡng và phạm vi truy vấn duplicate

Trong regression manual trước đó, UI đã hiển thị candidate similarity 91% và 89% với threshold 0.85. Việc quyết định merge vẫn thuộc người dùng; thao tác merge giữ canonical Test Case, chuyển source sang REJECTED và ghi version/audit, đúng mô hình human-in-the-loop.

4.6. Đóng gói & triển khai

Bản final chạy local và đã được rehearsal trên một thư mục fresh clone độc lập. PostgreSQL/pgvector và RabbitMQ là prerequisite của máy; phần còn lại được script trong repository chuẩn hóa. Quy trình được kiểm lại từ clone sạch đến đăng nhập thành công, không phụ thuộc ổ D:, PyCharm hay Docker.

Quick Start final trong README gồm bốn lệnh chính sau (sau khi prerequisite đã sẵn sàng và DATABASE_URL khớp môi trường local):

```
1) git clone -b overall https://github.com/NguyenPhuongQuyen/ai-assisted-test-case-generation.git && cd ai-assisted-test-case-generation
2) ./scripts/install.sh
3) ./scripts/init-demo.sh
4) ./scripts/dev-up.sh
```

Repository bàn giao: https://github.com/NguyenPhuongQuyen/ai-assisted-test-case-generation. Nhánh đang chốt báo cáo là overall, commit 2521bd8. Bản hiện tại được nghiệm thu local; cloud/HTTPS chưa triển khai và được ghi nhận là hạn

chế/hướng phát triển, không được mô tả như chức năng đã hoàn thành. Trước buổi bảo vệ cần gắn tag v1.0 vào đúng commit source final theo DP-06.

Biến môi trường	Ý nghĩa	Ghi chú bảo mật
DATABASE_URL	Kết nối PostgreSQL	Không ghi connection string thật vào báo cáo/repo.
JWT_SECRET / JWT_EXPIRE_MINUTES	Ký và giới hạn thời gian JWT	Secret chỉ lấy từ environment.
OPENAI_API_KEY	Khóa provider	Không commit; cần cho AI generation/embedding. Luồng manual-create không gọi provider.
OPENAI_MODEL / embedding config	Model generation và embedding	Model runtime ưu tiên active PromptConfig.
DUPLICATE_SIMILARITY_THRESHOLD	Ngưỡng duplicate	Mặc định 0.85.
AI_RATE_LIMIT_*	Giới hạn số generation request	Demo từng chạm 10 request / 300 giây theo cấu hình.
CELERY_BROKER_URL	RabbitMQ broker	Local guest chỉ dùng môi trường phát triển.
NEXT_PUBLIC_API_URL	Base URL frontend gọi backend	Local: http://127.0.0.1:8001/api/v1.

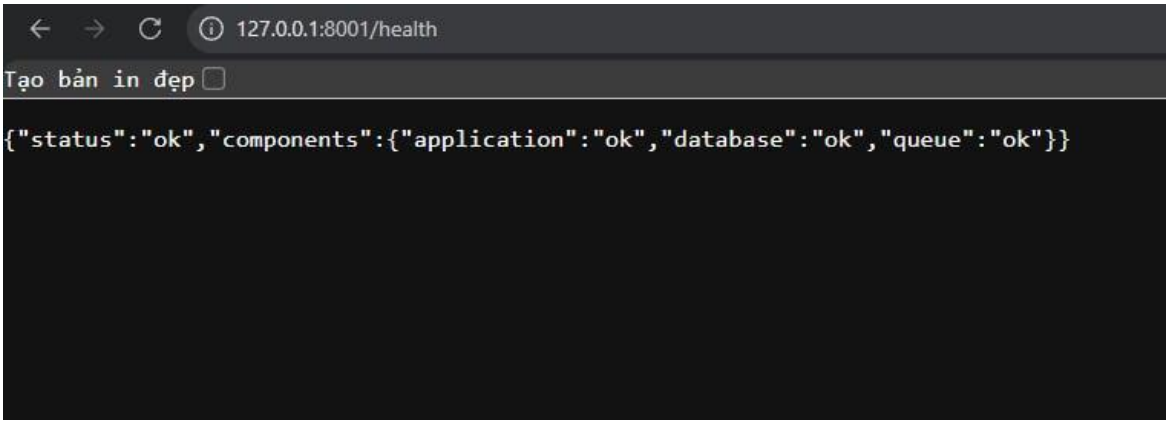
Bảng 4.10: Biến môi trường cần cho triển khai

Role	Email demo	Password demo mặc định
Admin	admin@example.com	Demo_Change_Me_123!
Manager	manager@example.com	
QA	qa@example.com	

Bảng 4.11: Tài khoản demo sau khi chạy seed

Fresh-clone rehearsal ngày 16/09/2026 đã xác nhận: install.sh tự tạo .env, .env.local, .venv và cài dependency; init-demo.sh migration + seed thành công; dev-up.sh khởi động hệ thống; QA đăng nhập được bằng tài khoản demo. Source chức năng được kiểm trên commit dbcd0d4; commit 2521bd8 sau đó chỉ hoàn thiện README và bộ ảnh minh họa. Tag v1.0 cần trỏ tới bản source final này trước khi nộp bảo vệ.

Hình 4.9 là health check của bản final. Endpoint /health không trả 'ok' cố định mà kiểm tra application, PostgreSQL và RabbitMQ; khi dependency quan trọng không hoạt động, API trả trạng thái degraded/HTTP 503. Ảnh final cho thấy cả ba thành phần đều ở trạng thái ok.



Hình 4.9: Health check xác nhận application, database và queue hoạt động

GitHub Actions vẫn là quality gate của repository. Hình 5.1 giữ bằng chứng CI run #91 ở baseline Tuần 09; sau các thay đổi handoff cuối, quality gate local được chạy lại với 110 backend test PASS, Ruff PASS và frontend Prettier/ESLint/Next.js build PASS. Báo cáo không dùng run #91 như số liệu 'mới nhất' của bản final.

CHƯƠNG 5: KIỂM THỬ & ĐÁNH GIÁ

Chương 5 báo cáo bằng chứng kiểm thử thực tế và tách rõ bằng chứng lịch sử với quality gate final. Kết quả chỉ ghi PASS khi có execution evidence; các Test Case chưa chạy đầy đủ được giữ trạng thái chưa kiểm. Bản handoff cuối bổ sung integration cho manual Test Case và concurrent export, sau đó chạy lại toàn bộ backend và frontend trước khi fresh-clone rehearsal.

5.1. Chiến lược kiểm thử

Mức/loại	Phạm vi	Môi trường/dữ liệu	Tiêu chí hoàn tất
Unit	Service rule, state transition, adapter boundary, export/duplicate/version	pytest; mock OpenAI/embedding theo TE-18	Rule có happy/negative và regression cho defect Major.
Integration	Auth, Requirement/BR08, generation, HITL, duplicate merge, export, manual-create, concurrent export	PostgreSQL riêng testcase_ai_test; migration thật	API + DB state đúng; không dùng DB demo.
API smoke	Health, login, create Requirement, submit job, 404 module, 401 no token	Newman 6.2.2; local services đang chạy	0 request/assertion failure.
Manual UI/E2E	Role navigation, Requirement, AI job, manual-create, HITL, Coverage, export, user/admin/version, fresh clone/login	testcase_ai demo + RabbitMQ/Celery + OpenAI thật cho Gold Set	Luồng đại diện chạy được, không ghi thay cho TC chưa chạy.
AI quality	20 Requirement Gold Set	OpenAI API thật; dữ liệu mẫu tự gán nhãn	Schema + Coverage/technique/hallucination được thống kê riêng.
CI	Backend + frontend trên PR/push	GitHub Actions baseline; final handoff còn được chạy local trên Windows Git Bash	Backend/frontend jobs success trước merge.

Bảng 5.1: Chiến lược, môi trường và tiêu chí dừng kiểm thử

Phạm vi ưu tiên bám BRD: BR-01–BR-09, authorization theo role/record, generation bất đồng bộ, version/audit, duplicate, Coverage và export. Ở lượt final, integration còn kiểm manual-create không gọi AI và concurrent export để tránh hai request cùng export một tập record. Test Case đầy đủ tiếp tục được quản lý trong docs/kiem-thu/.

5.2. Thiết kế Test Case

Test Case được thiết kế theo Equivalence Partitioning, Boundary Value Analysis, Decision Table, State Transition và authorization testing tùy rule. Bảng 5.2 là bảng rút gọn theo mẫu F.2; các dòng defect giữ kết quả FAIL ở lượt phát hiện và PASS ở lượt retest để tránh báo cáo "toàn Pass" sau khi đã sửa.

Mã TC	Chức năng	Đầu vào/tình huống	Kết quả mong đợi	KQ thực tế	TT
TC-GEN-007	UC06 / BR-04	AI trả output sai schema	Retry tối đa 2 lần; hết retry → AI_OUTPUT_INVALID 502	Đúng	Pass
TC-REQ-017	UC05 / BR-08	Update Requirement có TC approved/exported	TC liên quan → NEEDS_FIX + version/audit	Đúng	Pass
TC-AUTHZ-001	RBAC/record	User gọi endpoint/record không thuộc quyền	HTTP 403, không lộ dữ liệu	Đúng	Pass
TC-EXP-004	UC09 / BR-05	User không có quyền export	HTTP 403; không tạo file	Đúng	Pass
TC-T09-DF-001	AP-08 / Celery	Nhiều generation job liên tiếp	Worker xử lý ổn định, không lỗi event loop	Lượt 1 lỗi; retest PASS (#24)	Pass sau fix
TC-T09-	TE-16	Integration	Guard phải	Lượt 1	Pass sau fix

Mã TC	Chức năng	Đầu vào/tình huống	Kết quả mong đợi	KQ thực tế	TT
DF-002		trỏ DB demo	chặn; test chỉ chạy testcase_ai_test	chưa tách; retest PASS (#25)	
TC-T09-DF-003	UC05 / BR-08 UI	Rời workspace rồi mở Requirement đã lưu	QA chọn lại Requirement và cập nhật được	Lượt 1 FAIL; retest PASS (#26)	Pass sau fix

Bảng 5.2: Test Case rút gọn và kết quả thực tế

Ba Test Case regression của defect được trình bày đủ chín trường dưới đây; các bộ test đầy đủ còn lại nằm trong docs/kiem-thu/ để giữ báo cáo chính không quá dài.

Trường	Nội dung
Mã TC	TC-T09-DF-001
Chức năng / UC	UC06 / AP-08
Mục tiêu	Chứng minh Celery xử lý nhiều generation job liên tiếp mà không lỗi event loop.
Tiền điều kiện	RabbitMQ/Celery/PostgreSQL chạy; có Requirement hợp lệ.
Dữ liệu đầu vào	Gửi liên tiếp nhiều generation job cho Requirement.
Các bước	1. Start worker. 2. Submit nhiều job. 3. Chờ trạng thái cuối. 4. Kiểm log và DB.
Kết quả mong đợi	Các job hoàn tất/failed nghiệp vụ đúng; worker không crash do event loop.
Kết quả thực tế	Lượt phát hiện: xuất hiện lỗi event loop trên Windows. Sau fix persistent asyncio.Runner: focused test + E2E PASS.
Trạng thái	PASS sau retest; Issue #24; fix 10b9ea0.

Bảng 5.3: Test Case đủ 9 trường – TC-T09-DF-001

Trường	Nội dung
Mã TC	TC-T09-DF-002
Chức năng / UC	TE-16 / Integration
Mục tiêu	Chứng minh integration không được chạy trên database demo.
Tiền điều kiện	Có testcase_ai (demo) và testcase_ai_test (test).
Dữ liệu đầu vào	Cấu hình integration với DATABASE_URL.
Các bước	1. Chạy integration suite. 2. Kiểm DB được fixture reset. 3. Thử trỏ nhầm demo để kiểm guard.
Kết quả mong đợi	Suite dùng testcase_ai_test; guard chặn testcase_ai.
Kết quả thực tế	Trước fix chưa tách rõ DB; sau fix 16 integration PASS và demo DB không bị reset.
Trạng thái	PASS sau retest; Issue #25; fix 1088e72.

Bảng 5.4: Test Case đủ 9 trường – TC-T09-DF-002

Trường	Nội dung
Mã TC	TC-T09-DF-003
Chức năng / UC	UC05 / BR-08
Mục tiêu	Chứng minh QA có thể mở lại Requirement đã lưu và cập nhật từ UI.
Tiền điều kiện	QA đã tạo Requirement; đăng nhập đúng owner.
Dữ liệu đầu vào	Chuyển workspace rồi quay lại Requirement & AI.
Các bước	1. Chọn Module. 2. Chọn Requirement đã lưu. 3. Kiểm SRS/AC/lock_version load. 4. Cập nhật.
Kết quả mong đợi	Requirement cũ được tải lại; update thành công; TC liên quan áp dụng BR-08.
Kết quả thực tế	Trước fix UI không có danh sách Requirement đã lưu. Sau fix API + dropdown + manual REQ #5 PASS.

Trường	Nội dung
Trạng thái	PASS sau retest; Issue #26; fix 0064144.

Bảng 5.5: Test Case đủ 9 trường – TC-T09-DF-003

5.3. Kiểm thử tự động & CI

Automated test được chạy cả local và CI. Unit test mock provider; integration dùng PostgreSQL test thật; CI không cần OpenAI thật. Newman dùng collection docs/kiem-thu/Tuan09_Newman.postman_collection.json để kiểm tra network-level API smoke.

Bằng chứng	Kết quả mới nhất	Ghi chú
Backend full local	110 PASS, 2 warnings	Final local 16/09/2026; 2 deprecation warnings từ FastAPI/Starlette, không fail.
Unit subset	92 PASS	Service/adapter/business rule.
Integration local	18 PASS, 2 warnings	DB testcase_ai_test; gồm manual-create và concurrent export.
Newman	6 request PASS; 6 assertion PASS; 0 fail	952 ms; avg 78 ms; min 3 ms; max 380 ms.
CI run #91 backend (baseline)	108 passed, 2 warnings	Bằng chứng lịch sử Tuần 09; trước 2 integration test bổ sung ở handoff.
Frontend final local	Prettier PASS; ESLint PASS; Next.js build PASS	Node 24.19.0; npm 11.17.0; Next.js 16.2.11.

Bảng 5.6: Kết quả kiểm thử tự động, API smoke và CI

Quality gate final ngày 16/09/2026:

Backend: 110 passed, 2 warnings in 15.96s
Unit: 92 passed
Integration: 18 passed, 2 warnings in 13.70s
Ruff: 122 files already formatted; All checks passed
Frontend: Prettier PASS; ESLint PASS; Next.js production build PASS
Fresh clone: install → migration/seed → dev-up → login PASS
CI run #91: 108 passed, 2 warnings (baseline lịch sử trước hai integration test handoff)

Hình 5.1 được giữ như bằng chứng workflow CI đã từng chạy xanh ở baseline Tuần 09. Số liệu dùng để kết luận bản final là quality gate local 110 PASS/18 integration PASS nêu trong Bảng 5.6; hai warning còn lại đều là deprecation warning của dependency, không phải test failure.

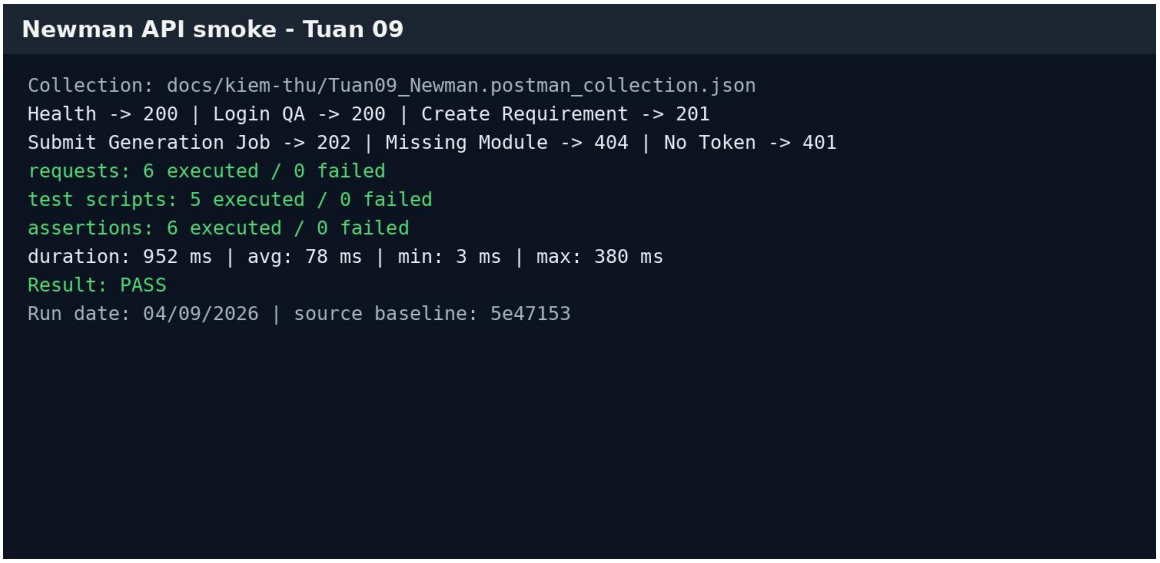
Hình 5.1 là log backend-quality của GitHub Actions run #91 ở baseline Tuần 09. Điểm cần nhìn là pipeline migration + pytest chạy thật trên database test và kết thúc xanh; ảnh này không được dùng để thay thế số liệu final 110 PASS được chạy lại local sau handoff.



Hình 5.1: GitHub Actions run #91 – baseline 108 backend test PASS

Hình 5.2 ghi kết quả Newman smoke thực tế đã chạy ngày 04/09/2026. Bộ smoke xác nhận sáu API request trọng yếu/âm có status code đúng và toàn bộ sáu

assertion đều PASS; các nghiệp vụ sâu hơn được phủ bởi pytest integration và manual regression.



Hình 5.2: Kết quả Newman API smoke Tuần 09

5.4. Ma trận truy vết yêu cầu → Test Case

Bảng 5.7 sử dụng cùng hệ mã Test Case với docs/kiem-thu/matran-truyvet.md và bổ sung kết quả của lượt handoff final. Cột kết quả giữ nguyên các chỗ chưa chạy đầy đủ; yêu cầu không có execution evidence không được tự động ghi PASS.

Mã YC/BR	Nội dung	Test Case phủ	Kết quả lượt gần nhất
YC-01	Nhập/cập nhật Requirement	TC-REQ-012…017	Unit/Integration PASS; một số BVA manual chưa chạy.
YC-02	AI generation + structured output	TC-GEN-004…007	PASS – Unit/Integration + Real AI retest.
YC-03	HITL + duplicate	TC-REV-001…007; TC-DUP-001…005	PASS – Unit/Integration + manual HITL/duplicate candidate.
YC-04	Module/tag/Covera	TC-MOD-	PASS –

Mã YC/BR	Nội dung	Test Case phủ	Kết quả lượt gần nhất
	ge/export	001…005; TC-COV-001…003; TC-EXP-001…005	Unit/Integration + manual Coverage/CSV/XLSX.
YC-05	Version/prompt/auth/user/audit	TC-VER/PROMPT/AUTH/USER/AUTHZ	PASS – Unit/Integration + manual Admin/User.
BR-01	AI output phải human review	TC-REV-002, TC-APP-002, TC-EXP-003	PASS.
BR-02	Đủ summary/steps/expected/priority	TC-APP-001,004; TC-GEN-006	PASS.
BR-03	Gắn module + Requirement nguồn	TC-APP-001, TC-GEN-004	PASS.
BR-04	Validate schema + regenerate	TC-GEN-007	PASS – Unit/Regression.
BR-05	Quyền approve/export	TC-APP-003, TC-EXP-004, TC-AUTHZ-001…003	PASS.
BR-06	Version + audit generate/edit/approve	TC-GEN-004, TC-REV-001, TC-APP-001	PASS.
BR-07	Phân quyền Requirement/Test Case	TC-REQ-015, TC-VER-005, TC-GEN-005, TC-AUTHZ-001	PASS.
BR-08	Requirement đổi → TC cần review	TC-REQ-017	PASS – Unit/Integration + UI regression.
BR-09	Boundary/high-priority quality	TC-REQ-012…014; TC-APP-001; Gold Set	Priority PASS; boundary manual chưa đủ nhãn để kết

Mã YC/BR	Nội dung	Test Case phủ	Kết quả lượt gần nhất
		technique rubric	luận KPI Boundary Coverage.

Bảng 5.7: Ma trận truy vết YC/BR → Test Case → kết quả gần nhất

5.5. Kết quả & defect

Hai lượt dưới đây được dùng để thể hiện bước tiến từ baseline Tuần 09 sang bản handoff cuối. Full backend tăng từ 108 lên 110 test và integration tăng từ 16 lên 18 sau khi bổ sung regression cho manual-create và concurrent export. Bảng defect riêng vẫn giữ FAIL ở lượt phát hiện và PASS sau retest để không tạo cảm giác 'toàn Pass từ đầu'.

Lượt chạy	Full backend	Integration	Manual/E2E nổi bật	Kết luận
Lượt 1 – Tuần 09 baseline	108 PASS	16 PASS	Requirement reopen, HITL, export, version, Admin/User, Gold Set	Regression đại diện + AI E2E PASS.
Lượt 2 – Final handoff 16/09	110 PASS	18 PASS	Manual-create, concurrent export, health degraded, fresh clone/login	Final local quality gate + fresh clone PASS.

Bảng 5.8: So sánh hai lượt kiểm thử tổng thể

Lượt	Tổng TC defect	Pass	Fail	Blocked	Tỉ lệ Pass
Lượt phát hiện	3	0	3	0	0%
Lượt retest sau fix	3	3	0	0	100%

Bảng 5.9: Pass/Fail hai lượt cho bộ regression defect Tuần 09

Issue	Severity	Module	Kết quả phát hiện	Fix/Retest	Trạng thái kỹ thuật cuối
#24	Major	Celery / async DB	Worker có thể lỗi event loop khi nhiều job	10b9ea0; automated + E2E PASS	Fixed; RETEST PASS; code đã merge
#25	Major	Test isolation / PostgreSQL	Integration có nguy cơ dùng DB demo	1088e72; 16 integration PASS + guard	Fixed; RETEST PASS; code đã merge
#26	Major	Requirement UI/API	QA không mở lại được Requirement đã lưu	0064144; API + UI regression PASS	Fixed; RETEST PASS; code đã merge

Bảng 5.10: Danh sách defect gắn GitHub Issue và regression

Mức nghiêm trọng	Phát hiện	Đã sửa + retest	Chưa xử lý	Ghi chú
critical	0	0	0	Không có.
major	3	3	0	Cả 3 defect Major đã có fix và regression PASS.
minor	0	0	0	Không ghi nhận trong bộ defect Tuần 09.
trivial	0	0	0	Không ghi nhận.

Bảng 5.11: Thống kê defect theo mức nghiêm trọng tại thời điểm chốt báo cáo

Ba defect Major #24–#26 đều đã có fix, automated/manual regression phù hợp và retest PASS. Báo cáo kết luận theo trạng thái kỹ thuật của defect; trạng thái

Open/Closed của GitHub Issue chỉ là metadata quản lý công việc và không được dùng thay cho execution evidence.

5.6. Đánh giá đối chiếu mục tiêu Chương 1

Gold Set gồm 20 Requirement được chạy ở snapshot đánh giá ngày 04/09/2026 qua FastAPI → RabbitMQ → Celery → OpenAI API → Structured Output validation → PostgreSQL. Kết quả snapshot là 20/20 generation hoàn thành và tạo 110 Test Case. Sau các thay đổi handoff, nhóm Gold Set không được chạy lại toàn bộ; vì vậy Bảng 5.12–5.13 giữ vai trò bằng chứng AI-quality tại thời điểm đo, còn quality gate final 16/09 tập trung vào code/integration/reproducibility.

Mục tiêu Chương 1	Cách đo	Số đo được	Đạt?	Nhận xét
Schema validity ≥95%	Completed generation parse đúng schema / tổng completed	20/20 = 100% (snapshot 04/09)	Đạt	BR-02/BR-04 được Structured Output + validation bảo vệ.
Human acceptance ≥80%	TC được duyệt với tối đa chỉnh sửa nhỏ / TC AI sinh	Chưa có dataset review đủ 110 TC	Chưa kết luận	Không suy diễn từ Schema Valid.
Giảm thời gian ≥50%	So thời gian thủ công và dùng hệ thống trên cùng Requirement	Chưa benchmark có đồng hồ	Chưa kết luận	Cần đo generation duration và thời gian review.
Boundary Coverage ≥90%	Boundary condition có TC / tổng boundary label Gold Set	Chưa gán nhãn boundary riêng	Chưa kết luận	Technique Match 100% không thay thế Boundary Coverage.
Traceability 100%	TC có requirement_id + module_id / tổng TC	Schema/DB bắt buộc với TC đã lưu	Đạt về cấu trúc	RTM vẫn ghi rõ boundary manual chưa chạy.

Mục tiêu Chương 1	Cách đo	Số đo được	Đạt?	Nhận xét
Export hợp lệ 100%	File CSV/XLSX mở được đúng cột / lần export đã kiểm	CSV PASS; XLSX PASS	Đạt trên mẫu đã kiểm	Không tuyên bố mọi tổ hợp export đều đã chạy.

Bảng 5.12: Đối chiếu KPI Chương 1 với số đo thực tế

KPI Gold Set bổ sung	Kết quả
Generation Success Rate	20/20 – 100%
Schema Valid Rate	20/20 – 100%
Full Coverage Rate	15/20 – 75%
Partial Coverage	5/20 – 25%
Coverage FAIL	0/20 – 0%
Technique Match Rate	20/20 – 100%
No Hallucination Rate	20/20 – 100%
Tổng Test Case AI sinh	110
Trung bình TC / Requirement	5.5

Bảng 5.13: Kết quả Gold Set 20 Requirement

Trong batch Gold Set ngày 04/09, GS11–GS20 từng nhận HTTP 429 vì rate limiter đạt giới hạn 10 generation request / 300 giây. Sau khi cửa sổ rate-limit reset, cùng Requirement được retry và hoàn tất đủ 20/20. Full Coverage 75% cho thấy Structured Output hợp lệ không đồng nghĩa với Coverage nghiệp vụ đầy đủ. Các chỉ số này không được tự động gán cho bản handoff 16/09 nếu chưa rerun Gold Set với cùng prompt/model/data.

5.7. Phần chưa kiểm được

Hạng mục chưa đủ bằng chứng	Vì sao chưa kết luận	Cách kiểm nếu tiếp tục
Human Acceptance Rate ≥80%	Chưa review/gán nhãn đủ 110 TC Gold Set theo tiêu chí chỉnh sửa nhỏ.	Manager/QA review toàn bộ mẫu, lưu approve/reject + version diff rồi tính tỉ lệ.
Boundary Coverage ≥90%	Gold Set chưa có danh sách boundary condition chuẩn để làm mẫu số.	Gán nhãn boundary/edge case cho từng Requirement rồi đối chiếu TC sinh.
Time Saving ≥50%	Chưa đo cùng Requirement bằng đồng hồ cho hai quy trình.	Ghi generation duration + thời gian review và benchmark viết thủ công.
Duplicate Rate tổng thể	Đã kiểm candidate/merge, chưa thống kê trên toàn 110 TC.	Rebuild embedding và chạy duplicate scan theo từng Module, thống kê candidate/merge.
Manual 128/128	Luồng manual-create đã có integration PASS; chưa có execution evidence thủ công cho toàn bộ 128 mã TC thiết kế.	Chạy theo test cycle cố định, điền actual/status/date/commit từng TC.
UI Loading/Empty/Error đầy đủ	Main-flow đã kiểm nhưng data-state chưa chạy hết mọi màn.	Thực thi TC-FE-001…005 và chụp evidence ở các state tương ứng.

Bảng 5.14: Phần chưa kiểm được và kế hoạch kiểm tiếp

CHƯƠNG 6: KẾT LUẬN & HƯỚNG PHÁT TRIỂN

6.1. Kết quả đạt được

Đề tài đã hiện thực chuỗi nghiệp vụ chính từ Requirement đến Test Case có kiểm soát của con người: nhập/cập nhật Requirement; sinh Test Case có Structured Output qua job bất đồng bộ; fallback tạo Test Case thủ công không gọi AI; HITL review/approve/needs-fix/reject; duplicate bằng pgvector; module/tag/Coverage;

version/compare/restore; export CSV/XLSX; authentication/RBAC/user administration; prompt/model version và audit.

Về kiểm thử, quality gate final đạt 110 backend tests PASS, trong đó 92 unit và 18 integration; frontend Prettier, ESLint và Next.js production build đều PASS. Fresh-clone rehearsal xác nhận install → migration/seed → start → login thành công trên thư mục độc lập. Newman 6/6 request và 6/6 assertion cùng CI run #91 được giữ làm bằng chứng lịch sử Tuần 09. Gold Set snapshot 04/09 hoàn thành 20/20 Requirement, tạo 110 Test Case; Schema Valid 100%, Full Coverage 75% và No Hallucination 100% theo rubric đã áp dụng.

So với mục tiêu Chương 1, chỉ tiêu Schema Valid ≥95% đạt; traceability được đảm bảo ở cấp cấu trúc dữ liệu/RTM; CSV/XLSX đã mở và kiểm được. Ba KPI Human Acceptance, Time Saving và Boundary Coverage chưa có phương pháp đo đủ dữ liệu nên báo cáo không kết luận đạt. Đây là kết luận dựa trên số đo ở Chương 5, không thay bằng nhận định "hệ thống đáp ứng đầy đủ".

Bản source dùng cho báo cáo được chốt ở commit 2521bd8 trên nhánh overall; fresh-clone đã kiểm trên source tương đương dbcd0d4 trước khi commit 2521bd8 chỉ bổ sung README và ảnh minh họa. Trước buổi bảo vệ cần gắn tag v1.0 vào đúng commit source final và kiểm lại quy trình Quick Start từ tag đó.

6.2. Hạn chế

Hạn chế	Nguyên nhân	Ảnh hưởng
Full Coverage Gold Set mới 75%	LLM vẫn có thể bỏ sót điều kiện nghiệp vụ dù schema đúng.	QA vẫn phải review; chưa thể dùng AI output trực tiếp.
Chưa đo Human Acceptance/Boundary/Time Saving	Thiếu nhãn review/boundary và benchmark có thời gian.	Chưa chứng minh định lượng đủ ba KPI Chương 1.
Gold Set mới 20 Requirement	Mẫu tối thiểu đáp ứng TE-18 nhưng còn hẹp về domain và độ khó.	Độ tin cậy khi khái quát sang dự án khác còn hạn chế.
Manual chưa chạy toàn bộ 128 TC	Ưu tiên automated/regression/E	Coverage thủ công toàn hệ thống chưa có bằng chứng

Hạn chế	Nguyên nhân	Ảnh hưởng
	2E trọng yếu; riêng manual-create đã có integration test.	đầy đủ.
Phụ thuộc provider và rate limit	Generation/embedding dùng API bên thứ ba; app có rate limiter.	Có độ trễ/chi phí và có thể nhận 429 khi burst request.
Phụ thuộc prerequisite local	Bản hiện tại đã có install/init/dev scripts, nhưng máy mới vẫn phải cài PostgreSQL/pgvector và RabbitMQ.	Khả năng dựng lại đã tốt hơn nhưng chưa đạt mức một lệnh/isolated runtime.

Bảng 6.1: Hạn chế, nguyên nhân và ảnh hưởng

6.3. Hướng phát triển

Mỗi hướng phát triển dưới đây gắn trực tiếp với một hạn chế đã nêu, tránh các đề xuất chung chung không liên quan tới kết quả hiện tại.

Hạn chế liên quan	Hướng phát triển cụ thể	Kết quả kỳ vọng
Coverage AI 75%	Mở rộng prompt/evaluation theo nhóm missing Coverage và thêm reviewer feedback vào bộ mẫu vàng.	Tăng Full Coverage mà vẫn giữ schema/hallucination gate.
KPI chưa đo	Lưu generation_duration, token/cost; gán nhãn accept/edit/reject và boundary cho Gold Set.	Tính được Human Acceptance, Time Saving, Boundary Coverage.
Gold Set 20 mẫu	Mở rộng nhiều module/domain, có easy/medium/hard và boundary label.	KPI ít phụ thuộc một nhóm Requirement.
Manual chưa đủ 128 TC	Tổ chức test cycle theo TC ID,	Có bằng chứng đầy đủ và giảm retest thủ công.

Hạn chế liên quan	Hướng phát triển cụ thể	Kết quả kỳ vọng
	actual/status/date/commit; tự động hóa thêm các case ổn định.	
Provider/rate limit	Thêm job metrics, retry/backoff cho lỗi provider phù hợp, dashboard queue và quota/cost.	Quan sát độ trễ, giảm failure tạm thời và kiểm soát chi phí.
Prerequisite local	Bổ sung docker-compose hoặc môi trường deploy HTTPS để đóng gói PostgreSQL/pgvector, RabbitMQ, backend và frontend.	Giảm phụ thuộc cấu hình thủ công và hỗ trợ demo/deploy nhất quán.

Bảng 6.2: Hướng phát triển gắn với hạn chế

6.4. Bài học rút ra

- Schema hợp lệ chỉ chứng minh cấu trúc, không chứng minh Coverage nghiệp vụ: Gold Set 100% Schema Valid nhưng Full Coverage 75%. Vì vậy BR-01/HITL là yêu cầu kiến trúc chứ không phải bước "duyệt cho có".
- Tách database test khỏi database demo phải được coi là điều kiện an toàn của test suite. Defect #25 cho thấy integration test đúng nghiệp vụ vẫn có thể gây rủi ro nếu fixture reset nhầm dữ liệu demo.
- Background worker cần quản lý vòng đời async resource nhất quán. Defect #24 giúp điều chỉnh worker theo persistent event loop/Runner thay vì tạo loop mới cho từng task trên Windows.
- Một rule backend chưa đủ để demo nghiệp vụ nếu UI không cho người dùng đi lại luồng đó. Defect #26 buộc bổ sung chức năng mở lại Requirement đã lưu, từ đó BR-08 có thể được kiểm trực tiếp trên giao diện.

TÀI LIỆU THAM KHẢO

[1] Võ Việt Khoa. BRD - Đề tài #13: Công cụ sinh Test Case tự động từ đặc tả yêu cầu bằng AI. ITEC4401, Trường Đại học Mở TP.HCM, 2026. *Tài liệu do GVHD cung cấp.*

[2] A. Mustafa et al., "Automated Test Case Generation from Requirements: A Systematic Literature Review," Computers, Materials & Continua, vol. 67, no. 2, pp. 1819-1833, 2021. DOI: 10.32604/cmc.2021.014391.

https://doi.org/10.32604/cmc.2021.014391

[3] A. Celik and Q. H. Mahmoud, "A Review of Large Language Models for Automated Test Case Generation," Machine Learning and Knowledge Extraction, vol. 7, no. 3, 97, 2025. DOI: 10.3390/make7030097.

https://doi.org/10.3390/make7030097

[4] S. Masuda et al., "Generating High-Level Test Cases from Requirements using LLM: An Industry Study," arXiv:2510.03641, 2025.

https://arxiv.org/abs/2510.03641

[5] A. M. Sami, Z. Rasheed, M. Waseem, Z. Zhang, H. Tomas, and P. Abrahamsson, "A Tool for Test Case Scenarios Generation Using Large Language Models," arXiv: 2406.07021, 2024.

https://arxiv.org/abs/2406.07021

[6] TestRail Support Center, "Quick Start: Generate Test Cases with AI," updated 23 Jan 2026.

https://support.testrail.com/hc/en-us/articles/37119835854484-Quick-Start-Generate-Test-Cases-with-AI

[7] Katalon Docs, "Generate Test Cases & Test Steps with AI," updated Apr 2026.

https://docs.katalon.com/katalon-platform/create-tests/generate-test-cases-with-ai

[8] testRigor, "AI-Based Test Automation Tool," product documentation/site, accessed Aug 2026.

https://testrigor.com/

[9] ISTQB, "Certified Tester Foundation Level Syllabus v4.0.1," 2024/2026 publication package.

https://istqb.org/wp-content/uploads/2024/11/ISTQB_CTFL_Syllabus_v4.0.1.pdf

[10] OpenAI, "Structured model outputs," OpenAI API Documentation, accessed Aug 2026.

https://developers.openai.com/api/docs/guides/structured-outputs

[11] A. Wright, H. Andrews, B. Hutton, and G. Dennis, "JSON Schema: A Media Type for Describing JSON Documents," Draft 2020-12, 2022.

https://json-schema.org/draft/2020-12/draft-bhutton-json-schema-01

[12] JSON Schema, "JSON Schema Validation: A Vocabulary for Structural Validation of JSON," Draft 2020-12, 2022.

https://json-schema.org/draft/2020-12/json-schema-validation

[13] OpenAI, "Prompting," OpenAI API Documentation, accessed Aug 2026.

https://developers.openai.com/api/docs/guides/prompting

[14] S. Amershi et al., "Guidelines for Human-AI Interaction," CHI 2019, pp. 1-13. DOI: 10.1145/3290605.3300233.

https://doi.org/10.1145/3290605.3300233

[15] M. W. Whalen, A. Rajan, M. P. E. Heimdahl, and S. P. Miller, "Coverage metrics for Requirements-based testing," ISSTA 2006, pp. 25-36. DOI: 10.1145/1146238.1146242.

https://doi.org/10.1145/1146238.1146242